Numerical Analysis

Yifan Li

August 2026

Contents

Introduction **4**

1 Errors, Floating-Point Arithmetic, and Numerical Stability **5**
1.1 Sources of Error in Numerical Computation 6
1.2 Binary Machine Numbers and Floating-Point Representation 6
1.3 Chopping, Rounding, and Significant Digits 7
1.4 Floating-Point Arithmetic 8
1.5 Reducing Round-Off Error 8
1.6 Summary of Numerical Error Principles 10

2 Nonlinear Equations and Iterative Root-Finding Methods **11**
2.1 Root-Finding Problems and the Bisection Method 12
2.2 Fixed-Point Iteration 13
2.3 Newton's Method 14
2.4 Order of Convergence 14
2.5 The Secant Method and Comparison of Root-Finding Methods 15

3 Interpolation and Piecewise Polynomial Approximation **17**
3.1 Polynomial Approximation and Interpolation 18
3.2 Lagrange Interpolation and the Error Formula 18
3.3 Neville's Method and Divided Differences 19
3.4 Equally Spaced Nodes and Hermite Interpolation 21
3.5 Piecewise Polynomial Approximation and Cubic Splines 22

4 Numerical Differentiation and Quadrature **25**
4.1 Finite-Difference Approximations of Derivatives 26
4.2 General Difference Formulas and Second Derivatives 27
4.3 Round-Off Error and Finite-Difference Discretization 28
4.4 Interpolatory Quadrature and Newton–Cotes Formulas 30
4.5 Composite Quadrature and Round-Off Stability 31
4.6 Gaussian Quadrature and Further Applications 32

5 Numerical Methods for Ordinary Differential Equations **35**
5.1 Initial Value Problems and the Forward Euler Method 36
5.2 Global Error Analysis of the Euler Method 36
5.3 Explicit and Implicit One-Step Methods 38
5.4 Runge–Kutta Methods 39
5.5 Consistency, Stability, and Convergence 40

6 Direct Methods for Linear Systems **41**
6.1 Gaussian Elimination and Backward Substitution 42
6.2 Pivoting and Round-Off Stability 42
6.3 Operation Counts and LU Decomposition 43
6.4 Permutation Matrices and Pivoted LU Factorization 44
6.5 Special Matrix Structures 44

7 Iterative Methods for Linear Systems **47**
7.1 Vector Norms and Matrix Norms 48

7.2 Eigenvalues, Spectral Radius, and Convergent Matrices 49
7.3 Residuals and Condition Numbers 50
7.4 Stationary Iterations and the Jacobi Method 51
7.5 Gauss–Seidel Method and Convergence Estimates 52
7.6 Successive Over-Relaxation 53

8 Eigenvalue Problems and Matrix Decompositions 54
8.1 Eigenvalue Problems and Localization 55
8.2 Power Methods, Shifts, and Rayleigh Quotients 56
8.3 Householder Transformations and Matrix Reduction 58
8.4 QR Factorization and the QR Eigenvalue Algorithm 59
8.5 Singular Value Decomposition 60
8.6 Applications of the Singular Value Decomposition 62

9 Least-Squares Approximation and Orthogonal Polynomials 64
9.1 Data Fitting and Least-Squares Problems 65
9.2 Normal Equations and Weighted Least Squares 66
9.3 Continuous Least-Squares Approximation 67
9.4 Orthogonal Bases and Gram–Schmidt Construction 68
9.5 Legendre and Chebyshev Polynomials 70

Acknowledgements 73

Introduction

Numerical analysis studies how mathematical problems can be transformed into finite computational procedures, and how the errors introduced by this transformation can be understood and controlled. In many problems, an exact solution may exist theoretically but cannot be obtained in a useful closed form. The objective is therefore not merely to compute an approximation, but to understand why the approximation is reliable, how accurate it is, and how its accuracy changes with computational cost.

A central theme is the passage from a continuous or exact problem to a discrete and computable one. Derivatives are replaced by difference quotients, integrals by weighted sums, differential equations by finite recurrence relations, and infinite-dimensional approximation problems by finite systems of algebraic equations. Each discretization introduces error, so numerical computation naturally leads to the study of truncation error, round-off error, conditioning, stability, consistency, and convergence. These concepts provide a common language for comparing numerical methods.

The course begins with errors and floating-point arithmetic, which explain the limitations of finite-precision computation. It then develops numerical methods for nonlinear equations, interpolation, numerical differentiation, and quadrature. These topics illustrate a recurring principle: a useful numerical method must balance approximation accuracy with computational stability.

For differential equations, the same principle appears in a more dynamic form. A continuous initial value problem is replaced by a sequence of discrete time steps. Local truncation errors are generated at individual steps and then propagated through the computation, leading to the distinction between local and global error and to the fundamental relationship among consistency, stability, and convergence.

A substantial part of numerical analysis is also numerical linear algebra. Linear systems may be solved by direct methods such as Gaussian elimination, LU decomposition, and Cholesky factorization, or by iterative methods such as Jacobi, Gauss–Seidel, and successive over-relaxation. Matrix norms, spectral radius, and condition numbers provide the theoretical tools for analyzing their behavior.

The same linear-algebraic viewpoint leads naturally to eigenvalue problems and matrix decompositions. Power methods and QR iteration approximate eigenvalues, while orthogonal transformations improve both numerical stability and computational structure. Singular value decomposition extends spectral ideas to rectangular matrices and connects matrix computation with least-squares problems, low-rank approximation, and data reduction.

Finally, least-squares approximation and orthogonal polynomials return to the general problem of approximating complicated objects by simpler ones. Orthogonality converts coupled approximation problems into simpler independent calculations, while Legendre and Chebyshev polynomials reveal how the choice of basis and interpolation nodes can dramatically affect numerical accuracy.

Although these methods address different mathematical problems, they are organized by the same basic questions: How should the original problem be discretized? How large is the resulting error? Is the computation stable under perturbations? Does the numerical solution converge to the exact one as the discretization is refined? These questions form the conceptual framework of numerical analysis and connect the methods developed throughout these notes.

Chapter 1

Errors, Floating-Point Arithmetic, and Numerical Stability

1.1 Sources of Error in Numerical Computation

A numerical computation typically follows the chain

$$\text{Problems} \longrightarrow \text{Models} \longrightarrow \text{Algorithms} \longrightarrow \text{Programming} \longrightarrow \text{Results}.$$

Different types of errors may arise at different stages, including model error, approximation error, and round-off error. The computed results may also provide feedback to earlier stages of the numerical procedure.
The core of this course is the analysis of approximation error and round-off error.

Example 1.1.1. In exact arithmetic, $(\sqrt{3})^2 = 3$. On a computer, however, one may first obtain $\sqrt{3} \approx 1.732$, and then

$$1.732^2 = 2.999824.$$

The discrepancy is produced because a machine has finite memory and therefore performs finite-digit arithmetic.

Definition 1.1.2. The *actual error* of an approximation P^* to an exact quantity P is $P - P^*$. The *absolute error* is $|P - P^*|$, and, provided $P \neq 0$, the *relative error* is

$$\frac{|P - P^*|}{|P|}.$$

Thus, numerical analysis is concerned not only with constructing an approximation, but also with understanding how errors are generated and propagated through the computation.

1.2 Binary Machine Numbers and Floating-Point Representation

A fundamental question is: how is a number stored in a machine?
A binary machine number can be represented in the normalized form

$$(-1)^s 2^{c-1023}(1+f),$$

where $s \in \{0, 1\}$ is the sign bit,

$$c = \sum_{i=1}^{11} 2^{11-i} c_i, \qquad c_i \in \{0, 1\}, \qquad 0 \leq c \leq 2047,$$

is the exponent part, and

$$f = \sum_{i=1}^{52} \left(\frac{1}{2}\right)^i f_i, \qquad f_i \in \{0, 1\},$$

is the mantissa part.
Thus the machine representation consists schematically of

$$\boxed{\underbrace{s}_{\text{sign}} \quad \underbrace{c_1 c_2 \cdots c_{11}}_{\text{exponent}} \quad \underbrace{f_1 f_2 \cdots f_{52}}_{\text{mantissa}}}.$$

For standard double-precision floating-point arithmetic, the magnitude of representable finite numbers is approximately 1.798×10^{308}, corresponding to an exponent scale of approximately 2^{1024}.

The error associated with the last stored binary digit is of the order

$$2^{c-1023}\left(\frac{1}{2}\right)^{53} \approx 2^{c-1023} \times 10^{-16}.$$

Hence only about 16 significant decimal digits can be retained in double-precision arithmetic.

Remark 1.2.1. The important point is not the particular bit layout itself, but the fact that the set of machine numbers is finite. Most real numbers therefore cannot be represented exactly and must be replaced by nearby machine numbers.

1.3 Chopping, Rounding, and Significant Digits

Consider a decimal number written in normalized form as

$$y = 0.d_1 d_2 \cdots d_k d_{k+1} \cdots \times 10^n,$$

where $n \in \mathbb{Z}$, $d_1 \neq 0$, and $d_i \in \{0, 1, \ldots, 9\}$.
There are two standard ways to replace y by a finite-digit representation.

Definition 1.3.1. Under *chopping*,

$$\mathrm{fl}(y) = 0.d_1 d_2 \cdots d_k \times 10^n.$$

All digits after d_k are simply discarded.

Definition 1.3.2. Under *rounding*, one adds $5 \times 10^{n-(k+1)}$ and then chops the resulting number. This is the usual "round to the nearest digit" procedure.

Definition 1.3.3. Let P^* be an approximation to a nonzero quantity P. We say that P^* approximates P to t significant digits if

$$\frac{|P - P^*|}{|P|} \leq 5 \times 10^{-t},$$

where t is taken to be the largest integer for which the inequality holds.

For example, one significant digit corresponds to a relative tolerance of order 5×10^{-1}, whereas two significant digits correspond to a relative tolerance of order 5×10^{-2}.
Now suppose

$$y = 0.d_1 d_2 \cdots d_k d_{k+1} \cdots \times 10^n$$

and y is chopped after the k-th digit. Then

$$|y - \mathrm{fl}(y)| < 10^{n-k}.$$

Since $|y| \geq d_1\, 10^{n-1} \geq 10^{n-1}$, we have

$$\frac{|y - \mathrm{fl}(y)|}{|y|} < 10^{1-k}.$$

Therefore chopping after k digits guarantees at least $k-1$ significant digits. If $d_{k+1} < 5$, then the discarded part is smaller, and the chopped value has at least k significant digits.

1.4 Floating-Point Arithmetic

Even when the mathematical operands x and y are exact, a computer first replaces them by machine numbers and then rounds the result of the arithmetic operation.
For $\circ \in \{+, -, \times, \div\}$, define the corresponding floating-point operation by

$$x \circledast y = \mathrm{fl}(\mathrm{fl}(x) \circ \mathrm{fl}(y)) .$$

For example,

$$x \oplus y = \mathrm{fl}(\mathrm{fl}(x) + \mathrm{fl}(y)) , \qquad x \ominus y = \mathrm{fl}(\mathrm{fl}(x) - \mathrm{fl}(y)) ,$$

and similarly for floating-point multiplication and division.
Consequently, even a single elementary arithmetic operation may introduce round-off error, and a long sequence of such operations may accumulate or amplify these errors.

1.5 Reducing Round-Off Error

There are several common situations in which a mathematically equivalent reformulation can substantially reduce numerical error.

1.5.1 Avoid a Very Small Denominator

Suppose $\mathrm{fl}(z) = z + \delta$ and $\varepsilon = 10^{-m}$. Then division by the small quantity ε gives

$$z \oslash \varepsilon = \mathrm{fl}\left(\frac{\mathrm{fl}(z)}{\mathrm{fl}(\varepsilon)}\right) = \mathrm{fl}\left(\frac{z+\delta}{\varepsilon}\right) = \mathrm{fl}((z+\delta)10^{m}) .$$

Therefore the perturbation δ may be magnified to approximately $\Delta P = 10^{m}\delta$. Thus one should avoid numerical formulas in which the numerator is divided by an unnecessarily small denominator.

Example 1.5.1. Consider

$$\frac{e^{x} - 1}{x}$$

for $x = 0.01$. The exact value is approximately

$$\frac{e^{0.01} - 1}{0.01} \approx 1.0050167.$$

Since $e^{0.01} \approx 1.010050$, finite-digit chopping may replace this value by a shorter representation. The subtraction $e^{0.01} - 1$ then loses some of the available information, and division by 0.01 amplifies the resulting error. With sufficiently coarse chopping, the computed answer may even reduce to approximately 1, although the exact value is approximately 1.0050167.

1.5.2 Avoid Subtracting Nearly Equal Numbers

Suppose

$$\mathrm{fl}(x) = 0.d_1 d_2 \cdots d_p d_{p+1} \cdots d_k \times 10^{m}$$

and

$$\mathrm{fl}(y) = 0.d_1 d_2 \cdots d_p \beta_{p+1} \cdots \beta_k \times 10^{m}.$$

The first p digits agree. Then the subtraction $x \ominus y$ causes these common leading digits to cancel, so that only approximately $k - p$ significant digits remain.
This phenomenon is called *catastrophic cancellation*.

Example 1.5.2. Consider $100 - \sqrt{9999}$ using three-digit chopping. We have

$$\mathrm{fl}(\sqrt{9999}) = 0.994 \times 10^2, \qquad \mathrm{fl}(100) = 0.100 \times 10^3.$$

Direct subtraction gives

$$100 \ominus \sqrt{9999} = \mathrm{fl}\Big(0.100 \times 10^3 - 0.994 \times 10^2\Big) = \mathrm{fl}\Big(0.100 \times 10^3 - 0.0994 \times 10^3\Big) = \mathrm{fl}\Big(0.001 \times 10^3\Big) = 0.100$$

The resulting error is approximately 0.0499.
However,

$$100 - \sqrt{9999} = \frac{1}{100 + \sqrt{9999}},$$

so the mathematically equivalent expression gives

$$1 \oslash \Big(100 \oplus \sqrt{9999}\Big) = \mathrm{fl}\left(\frac{0.100 \times 10^1}{0.199 \times 10^3}\right) = \mathrm{fl}\Big(0.5025 \times 10^{-2}\Big) = 0.503 \times 10^{-2}.$$

The error is now approximately 0.0003. Thus an algebraically equivalent reformulation can dramatically improve numerical accuracy.

1.5.3 Avoid Repeated Addition of Tiny Numbers to a Large Number

If $x \gg y$, then the floating-point addition $x \oplus y$ may simply return x, because y is too small to affect the stored digits of x.

Example 1.5.3. Consider

$$10000 + 0.1 + 0.1 + \cdots + 0.1,$$

where 0.1 occurs 1000 times. If the additions are performed from left to right in low-precision arithmetic, each addition of 0.1 to 10000 may be lost, producing approximately 10000, with an error of 100.
Instead, first add the small quantities,

$$0.1 + 0.1 + \cdots + 0.1 = 100,$$

and then add the result to 10000. This ordering avoids the loss of the small increments.

Thus the order of arithmetic operations matters in floating-point computation even though it does not matter in exact arithmetic.

1.5.4 Reduce the Number of Arithmetic Operations

Reducing the number of arithmetic operations can both decrease computational cost and reduce opportunities for round-off error. Consider the polynomial

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0.$$

A direct evaluation requires

$$n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2}$$

multiplications to construct the powers separately, together with n additions.
Instead, use nested arithmetic:

$$f(x) = \left(\cdots \left((a_n x + a_{n-1})x + a_{n-2}\right) x + \cdots + a_1\right) x + a_0.$$

This requires only **n** multiplications and **n** additions.

Remark 1.5.4. This nested form is commonly called *Horner's method*. In addition to reducing the operation count from a quadratic number of multiplications to a linear number, it generally provides a numerically preferable way to evaluate a polynomial.

1.6 Summary of Numerical Error Principles

The first stage of numerical analysis already illustrates several principles that will recur throughout the course:

1. A computer performs finite-digit rather than exact arithmetic.
2. Every real number that is not exactly representable must first be approximated by a machine number.
3. Round-off errors may be amplified by later arithmetic operations.
4. Subtracting nearly equal numbers can destroy significant digits.
5. Adding a very small number to a much larger number may have no numerical effect.
6. Algebraically equivalent formulas may have very different numerical behavior.
7. Reducing the number of arithmetic operations can improve both efficiency and numerical reliability.

These observations motivate the broader concept of *numerical stability*: a good numerical algorithm should not unnecessarily amplify the small perturbations created by finite-precision arithmetic.

Chapter 2

Nonlinear Equations and Iterative Root-Finding Methods

2.1 Root-Finding Problems and the Bisection Method

A fundamental root-finding problem is to find x such that $f(x) = 0$. The simplest numerical method for this problem is the *bisection method*.

Theorem 2.1.1 (Intermediate Value Theorem)**.** Let $f \in C[a, b]$. For every k between $f(a)$ and $f(b)$, there exists $c \in [a, b]$ such that $f(c) = k$.

Corollary 2.1.2. If $f \in C[a, b]$ and $f(a)f(b) < 0$, then there exists $x_0 \in (a, b)$ such that $f(x_0) = 0$.

The basic idea of the bisection method is to repeatedly shrink an interval that contains a root.
Given an initial interval $[a_1, b_1]$, a maximum number of iterations N_0, and a tolerance $\varepsilon > 0$, the algorithm proceeds as follows.

1. Find a_1, b_1 such that $f(a_1)f(b_1) < 0$.
2. For $i = 1, 2, \ldots, N_0$, set
$$p_i = \frac{a_i + b_i}{2}.$$
3. If $|p_i - p_{i-1}| < \varepsilon$ or $|f(p_i)| < \varepsilon$, stop and output p_i.
4. If $f(p_i)f(a_i) > 0$, set $a_{i+1} = p_i$ and $b_{i+1} = b_i$. If $f(p_i)f(a_i) < 0$, set $a_{i+1} = a_i$ and $b_{i+1} = p_i$.
5. Set $i = i + 1$ and repeat the procedure.
6. If the maximum number of iterations is reached, stop and output "Failure".

Theorem 2.1.3 (Error Bound for Bisection)**.** Let p^* satisfy $f(p^*) = 0$, and let $p_n = (a_n + b_n)/2$ be the midpoint obtained after n bisections. Then

$$|p_n - p^*| \leq \frac{b - a}{2^n}.$$

Therefore

$$p_n = p^* + O(2^{-n}).$$

Proof. After each bisection, the length of the interval containing p^* is reduced by a factor of $1/2$. Hence after n steps its length is $(b-a)/2^n$. Since both p_n and p^* lie in this interval,

$$|p_n - p^*| \leq \frac{b - a}{2^n}.$$

□

To achieve accuracy 10^{-m}, it is sufficient to require $(b - a)/2^N \leq 10^{-m}$, or equivalently

$$N \geq \log_2(10^m(b - a)).$$

Remark 2.1.4. The advantages of the bisection method are that it requires only continuity of f and, once a sign-changing interval is found, it always converges. Its drawbacks are that convergence is slow, the midpoint itself is discarded after each decision, and the method is not convenient for high-dimensional problems.

2.2 Fixed-Point Iteration

Instead of solving $f(p) = 0$ directly, one may rewrite the equation in the form $p = g(p)$.

Definition 2.2.1. A point p is called a *fixed point* of g if $g(p) = p$. Starting from an initial value p_0, the *fixed-point iteration* is defined by

$$p_n = g(p_{n-1}), \qquad n = 1, 2, \ldots.$$

If $p_n \to p$ and g is continuous, then

$$p = \lim_{n\to\infty} p_n = \lim_{n\to\infty} g(p_{n-1}) = g\Big(\lim_{n\to\infty} p_{n-1}\Big) = g(p),$$

so the limit is necessarily a fixed point.

Theorem 2.2.2 (Existence of a Fixed Point)**.** If $g \in C[a, b]$ and $g([a, b]) \subset [a, b]$, then g has at least one fixed point in $[a, b]$; that is, there exists $p \in [a, b]$ such that $p = g(p)$.

Proof. Define $h(x) = g(x) - x$. Since $g([a, b]) \subset [a, b]$, $g(a) \geq a$ and $g(b) \leq b$. Hence $h(a) \geq 0$ and $h(b) \leq 0$. Since h is continuous, the Intermediate Value Theorem implies that there exists $p \in [a, b]$ such that $h(p) = 0$, and therefore $g(p) = p$. □

Theorem 2.2.3 (Uniqueness and Convergence)**.** Suppose $g \in C[a, b]$, $g([a, b]) \subset [a, b]$, and $g'(x)$ exists on (a, b). If there exists $0 < k < 1$ such that $|g'(x)| \leq k$ for every $x \in (a, b)$, then g has a unique fixed point $p \in [a, b]$. Moreover, for any $p_0 \in [a, b]$, the sequence defined by $p_n = g(p_{n-1})$ converges to p.

The iteration satisfies the error estimates

$$|p_n - p| \leq k^n \max\{p_0 - a, b - p_0\},$$

and, for $n \geq 1$,

$$|p_n - p| \leq \frac{k^n}{1 - k} |p_1 - p_0| .$$

Proof. By the Mean Value Theorem,

$$|p_n - p| = |g(p_{n-1}) - g(p)| = |g'(\xi_{n-1})| \, |p_{n-1} - p| \leq k \, |p_{n-1} - p| .$$

Repeating this estimate gives

$$|p_n - p| \leq k^n \, |p_0 - p| \leq k^n \max\{p_0 - a, b - p_0\}.$$

Since $0 < k < 1$, the right-hand side tends to zero, so $p_n \to p$.
For the second estimate,

$$|p_{n+1} - p_n| = |g(p_n) - g(p_{n-1})| \leq k \, |p_n - p_{n-1}| ,$$

and hence $|p_{n+1} - p_n| \leq k^n \, |p_1 - p_0|$. For $m > n$,

$$|p_m - p_n| \leq \sum_{j=n}^{m-1} |p_{j+1} - p_j| \leq |p_1 - p_0| \sum_{j=n}^{m-1} k^j.$$

Letting $m \to \infty$ gives

$$|p - p_n| \leq \frac{k^n}{1 - k} |p_1 - p_0| .$$

Uniqueness follows immediately: if p and q are fixed points, then $|p - q| = |g(p) - g(q)| \leq k \, |p - q|$, which is possible for $k < 1$ only when $p = q$. □

2.3 Newton's Method

Newton's method is obtained by constructing a fixed-point function whose derivative vanishes at the desired root.
Suppose $f(p) = 0$ and $f'(p) \neq 0$. Write

$$g(x) = x - \phi(x)f(x).$$

Then $g(p) = p$, and

$$g'(x) = 1 - \phi'(x)f(x) - \phi(x)f'(x).$$

At $x = p$, since $f(p) = 0$, $g'(p) = 1-\phi(p)f'(p)$. To obtain $g'(p) = 0$, choose $\phi(p) = 1/f'(p)$. Taking $\phi(x) = 1/f'(x)$ gives

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

Therefore Newton's iteration is

$$\boxed{p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}}.$$

Geometrically, Newton's method replaces the graph of f near p_{n-1} by its tangent line. The intersection of this tangent line with the x-axis gives p_n.

Example 2.3.1. For $f(x) = \cos x - x$ on $(0, \pi/2)$, $f'(x) = -\sin x - 1$. Hence Newton's iteration becomes

$$p_{n+1} = p_n - \frac{\cos p_n - p_n}{-\sin p_n - 1} = p_n + \frac{\cos p_n - p_n}{1 + \sin p_n}.$$

Theorem 2.3.2 (Local Convergence of Newton's Method)**.** Let $f \in C^2[a, b]$, and suppose $p \in (a, b)$ satisfies $f(p) = 0$ and $f'(p) \neq 0$. Then there exists $\delta > 0$ such that Newton's method generates a sequence converging to p for every $p_0 \in [p - \delta, p + \delta]$.

Thus Newton's method is a *local* method: convergence is guaranteed for initial guesses sufficiently close to a simple root, but an arbitrary initial guess need not converge.

2.4 Order of Convergence

To compare iterative methods quantitatively, we need a measure of their convergence speed.

Definition 2.4.1. Suppose $p_n \to p$ and $p_n \neq p$. If there exist $\alpha > 0$ and $\lambda > 0$ such that

$$\lim_{n\to\infty} \frac{|p_{n+1} - p|}{|p_n - p|^\alpha} = \lambda,$$

then $\{p_n\}$ is said to converge to p with *order* α and *asymptotic error constant* λ.

If $\alpha = 1$ and $0 < \lambda < 1$, the convergence is called *linear*. If $\alpha = 2$, the convergence is called *quadratic*.
Roughly speaking, if a linearly convergent sequence has error of size 10^{-an}, then the number of correct digits grows linearly with n. For a quadratically convergent sequence, the number of correct digits may grow approximately exponentially.

Theorem 2.4.2 (Linear Convergence of Fixed-Point Iteration)**.** Suppose $g \in C[a,b]$, $g([a,b]) \subset [a,b]$, g' is continuous on (a,b), and $|g'(x)| \leq k < 1$. Let p be the fixed point and suppose $g'(p) \neq 0$. Then the sequence generated by $p_n = g(p_{n-1})$ converges linearly to p.

Proof. By the Mean Value Theorem, for some ξ_n between p_n and p,

$$p_{n+1} - p = g(p_n) - g(p) = g'(\xi_n)(p_n - p).$$

Hence

$$\frac{|p_{n+1} - p|}{|p_n - p|} = |g'(\xi_n)|.$$

Since $p_n \to p$, we have $\xi_n \to p$, and continuity of g' gives

$$\lim_{n\to\infty} \frac{|p_{n+1} - p|}{|p_n - p|} = |g'(p)|.$$

Therefore the convergence is linear. □

Theorem 2.4.3 (Quadratic Convergence of Fixed-Point Iteration)**.** Suppose $g(p) = p$, $g'(p) = 0$, and g'' is continuous with $|g''(x)| \leq M$ on an open interval containing p. Then there exists $\delta > 0$ such that, for $p_0 \in [p - \delta, p + \delta]$, the fixed-point iteration $p_n = g(p_{n-1})$ converges to p at least quadratically.

Proof. Taylor's theorem about p gives

$$g(x) = g(p) + g'(p)(x - p) + \frac{1}{2}g''(\xi)(x - p)^2.$$

Since $g(p) = p$ and $g'(p) = 0$,

$$g(x) - p = \frac{1}{2}g''(\xi)(x - p)^2.$$

Therefore

$$|p_{n+1} - p| \leq \frac{M}{2}|p_n - p|^2,$$

which gives at least quadratic convergence. □

Newton's method is precisely designed so that its fixed-point function satisfies $g'(p) = 0$. This explains its quadratic convergence near a simple root.

2.5 The Secant Method and Comparison of Root-Finding Methods

Newton's method requires evaluation of $f'(x)$. The *secant method* replaces the derivative by a finite-difference approximation:

$$f'(p_{n-1}) \approx \frac{f(p_{n-1}) - f(p_{n-2})}{p_{n-1} - p_{n-2}}.$$

Substituting this approximation into Newton's formula gives

$$\boxed{p_n = p_{n-1} - \frac{f(p_{n-1})(p_{n-1} - p_{n-2})}{f(p_{n-1}) - f(p_{n-2})}}.$$

Geometrically, instead of using the tangent line at one point, the secant method uses the line through $(p_{n-2}, f(p_{n-2}))$ and $(p_{n-1}, f(p_{n-1}))$. Its intersection with the x-axis gives the next approximation p_n.
The principal methods discussed in this chapter can be compared as follows.

1. **Bisection method.** It is simple and always convergent once a sign-changing interval is available, but its convergence is slow.
2. **Fixed-point iteration.** With a suitable choice of g, it can be faster than bisection and is usually linearly convergent. However, constructing an appropriate fixed-point function g is not always easy.
3. **Newton's method.** It has a uniform iteration formula and is quadratically convergent near a simple root. It requires differentiability, $f'(p) \neq 0$, and sufficient regularity to obtain quadratic convergence. It is also sensitive to the initial guess.
4. **Secant method.** It avoids explicit evaluation of f' by approximating the derivative with a secant slope, while retaining the basic geometric idea of Newton's method.

Remark 2.5.1. The methods illustrate a recurring trade-off in numerical analysis. Bisection sacrifices speed for robustness, whereas Newton's method achieves much faster local convergence at the cost of stronger assumptions and greater sensitivity to the initial value.

Chapter 3

Interpolation and Piecewise Polynomial Approximation

3.1 Polynomial Approximation and Interpolation

Theorem 3.1.1 (Weierstrass Approximation Theorem)**.** For every $f \in C[a,b]$ and every $\varepsilon > 0$, there exists a polynomial $P(x)$ such that

$$|f(x) - P(x)| < \varepsilon \qquad \text{for every } x \in [a,b].$$

The theorem shows that continuous functions on a closed interval can be approximated arbitrarily well by polynomials. In numerical computation, however, the function is often known only through finitely many values. This leads to the interpolation problem.
Given $n+1$ distinct points $x_0, x_1, \dots, x_n$, the polynomial interpolation problem is to find a polynomial $P(x)$ with $\deg P \le n$ such that $P(x_i) = f(x_i)$ for $i = 0, 1, \dots, n$.
Write

$$P(x) = a_0 + a_1 x + \cdots + a_n x^n.$$

The interpolation conditions give the linear system

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix}.$$

The coefficient matrix is the *Vandermonde matrix*, whose determinant is

$$\det V = \prod_{0 \le j < i \le n} (x_i - x_j).$$

Since the interpolation nodes are distinct, $\det V \neq 0$. Therefore the interpolating polynomial exists and is unique.

3.2 Lagrange Interpolation and the Error Formula

For the distinct nodes $x_0, x_1, \dots, x_n$, define the Lagrange basis polynomials

$$L_k(x) = \prod_{\substack{0 \le j \le n \\ j \neq k}} \frac{x - x_j}{x_k - x_j}, \qquad k = 0, 1, \dots, n.$$

They satisfy $L_k(x_i) = \delta_{ki}$, where

$$\delta_{ki} = \begin{cases} 1, & i = k, \\ 0, & i \neq k. \end{cases}$$

Therefore the polynomial

$$P(x) = \sum_{i=0}^{n} f(x_i) L_i(x)$$

satisfies $\deg P \le n$ and $P(x_i) = f(x_i)$ for $i = 0, 1, \dots, n$.
To derive the interpolation error formula, we first recall Rolle's theorem.

Theorem 3.2.1 (Rolle's Theorem)**.** If $f \in C[a,b]$, f' exists on (a,b), and $f(a) = f(b)$, then there exists $c \in (a,b)$ such that $f'(c) = 0$.

A useful generalization is the following: if f has $n+1$ distinct zeros $x_0, x_1, \dots, x_n$, then there exists a point ξ between them such that $f^{(n)}(\xi) = 0$.

Theorem 3.2.2 (Interpolation Error)**.** Let $f \in C^{n+1}[a,b]$, let $x_0 < x_1 < \cdots < x_n$ be interpolation nodes, and let $P(x)$ be the interpolating polynomial of degree at most n. Then for every $x \in [a,b]$, there exists $\xi(x) \in (a,b)$ such that

$$f(x) - P(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

Proof. For a fixed x, define

$$g(t) = f(t) - P(t) - \Big(f(x) - P(x)\Big) \frac{\prod_{i=0}^{n}(t - x_i)}{\prod_{i=0}^{n}(x - x_i)}.$$

Then $g(x) = 0$, and $g(x_i) = 0$ for $i = 0, 1, \ldots, n$. Thus g has $n+2$ zeros. By the generalized Rolle theorem, there exists $\xi(x)$ such that $g^{(n+1)}(\xi(x)) = 0$.
Since $\deg P \le n$, $P^{(n+1)} = 0$, and

$$\frac{d^{n+1}}{dt^{n+1}} \prod_{i=0}^{n} (t - x_i) = (n+1)!.$$

Hence

$$0 = f^{(n+1)}(\xi(x)) - \Big(f(x) - P(x)\Big) \frac{(n+1)!}{\prod_{i=0}^{n}(x - x_i)},$$

which gives the result. □

3.3 Neville's Method and Divided Differences

When a new interpolation point is added, recomputing the complete Lagrange formula is inconvenient. Neville's method constructs the interpolating polynomial recursively and is therefore easy to implement when more points are added.
Let $P_{i,j}(x)$ denote the interpolating polynomial through $x_i, x_{i+1}, \ldots, x_j$. Then

$$P_{i,j}(x) = \frac{(x - x_j)P_{i,j-1}(x) - (x - x_i)P_{i+1,j}(x)}{x_i - x_j}.$$

For example,

$$P_{0,1,2}(x) = \frac{(x - x_2)P_{0,1}(x) - (x - x_0)P_{1,2}(x)}{x_0 - x_2}.$$

The recursive structure can be visualized schematically as

$$\begin{array}{llll} x_0 & P_0 & P_{0,1} & P_{0,1,2} \\ x_1 & P_1 & P_{1,2} & \\ x_2 & P_2 & & \end{array}$$

and additional nodes can be incorporated without rebuilding all previously computed quantities.
Another useful polynomial basis is

$$1, \quad x - x_0, \quad (x - x_0)(x - x_1), \quad \ldots, \quad \prod_{j=0}^{n-1} (x - x_j).$$

Thus one may write

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n \prod_{j=0}^{n-1} (x - x_j).$$

To determine the coefficients, introduce divided differences.

Definition 3.3.1. The zeroth divided difference is $f[x_i] = f(x_i)$. The first divided difference is

$$f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}.$$

More generally,

$$f[x_i, x_{i+1}, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

The divided differences can be organized in a triangular table:

$$\begin{array}{llll} x_0 & f[x_0] & & \\ x_1 & f[x_1] & f[x_0, x_1] & \\ x_2 & f[x_2] & f[x_1, x_2] & f[x_0, x_1, x_2] \\ \vdots & \vdots & \vdots & \ddots \end{array}$$

Theorem 3.3.2 (Newton's Divided-Difference Formula)**.** The interpolating polynomial is

$$P(x) = f[x_0] + \sum_{k=1}^{n} f[x_0, x_1, \ldots, x_k] \prod_{j=0}^{k-1} (x - x_j).$$

Proof. Using the definition of divided differences,

$$f(x) = f[x_0] + f[x, x_0](x - x_0).$$

Next,

$$f[x, x_0] = f[x_0, x_1] + f[x, x_0, x_1](x - x_1).$$

Substituting recursively gives

$$\begin{aligned} f(x) = f[x_0] &+ f[x_0, x_1](x - x_0) \\ &+ f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots \\ &+ f[x_0, \ldots, x_n] \prod_{j=0}^{n-1} (x - x_j) + R_n(x), \end{aligned}$$

where

$$R_n(x) = f[x, x_0, \ldots, x_n] \prod_{j=0}^{n} (x - x_j).$$

Therefore the coefficients in the Newton basis are $a_k = f[x_0, x_1, \ldots, x_k]$. □

The remainder formula is consequently

$$f(x) - P(x) = f[x, x_0, \ldots, x_n] \prod_{j=0}^{n} (x - x_j).$$

One major advantage of the Newton form is that when one new node is added, only one new basis function and one new coefficient are required. Since the new basis function vanishes at all old nodes, the coefficients of the old basis functions remain unchanged.

Corollary 3.3.3. For distinct $x_0, \ldots, x_n$ and sufficiently smooth f, there exists $\xi \in (a, b)$ such that

$$f[x_0, x_1, \ldots, x_n] = \frac{f^{(n)}(\xi)}{n!}.$$

This follows because the coefficient of x^n in the Newton interpolating polynomial is $f[x_0, \ldots, x_n]$, while the generalized Rolle theorem gives a point ξ for which $f^{(n)}(\xi) = P^{(n)}(\xi)$.

3.4 Equally Spaced Nodes and Hermite Interpolation

Suppose the interpolation points are equally spaced with $h = x_{i+1} - x_i$ and $x_i = x_0 + ih$. Let $x = x_0 + sh$. Then $x - x_i = (s - i)h$, and Newton's formula becomes

$$P(x) = f(x_0) + \sum_{k=1}^{n} s(s-1)\cdots(s-k+1)h^k f[x_0, x_1, \ldots, x_k].$$

Define the forward difference by $\Delta f(x_k) = f(x_{k+1}) - f(x_k)$. Then

$$f[x_0, x_1, \ldots, x_k] = \frac{\Delta^k f(x_0)}{k! h^k}.$$

Hence

$$P(x) = \sum_{k=0}^{n} \binom{s}{k} \Delta^k f(x_0), \qquad \binom{s}{k} = \frac{s(s-1)\cdots(s-k+1)}{k!}.$$

Similarly, define the backward difference by $\nabla f(x_k) = f(x_k) - f(x_{k-1})$. If $x = x_n + sh$, then the interpolating polynomial may be written in a corresponding backward difference form.
Interpolation may also use derivative information.

Theorem 3.4.1 (Hermite Interpolation)**.** Given distinct nodes $x_0, x_1, \ldots, x_n$, the unique polynomial of least degree satisfying

$$H(x_j) = f(x_j), \qquad H'(x_j) = f'(x_j), \qquad j = 0, 1, \ldots, n,$$

has degree at most $2n + 1$ and can be written as

$$H(x) = \sum_{j=0}^{n} \left[f(x_j) H_j(x) + f'(x_j) \widehat{H}_j(x) \right],$$

where

$$H_j(x) = \left[1 - 2(x - x_j) L_j'(x_j) \right] L_j^2(x), \qquad \widehat{H}_j(x) = (x - x_j) L_j^2(x).$$

These basis functions satisfy

$$H_j(x_i) = \delta_{ij}, \qquad H_j'(x_i) = 0,$$

and

$$\widehat{H}_j(x_i) = 0, \qquad \widehat{H}_j'(x_i) = \delta_{ij}.$$

Proof. For $i \neq j$, x_i is a double zero of H_j, so write $H_j(x) = (ax + b)L_j^2(x)$. Since $H_j(x_j) = 1$ and $H_j'(x_j) = 0$,

$$ax_j + b = 1, \qquad a + 2(ax_j + b)L_j'(x_j) = 0.$$

Thus $a = -2L_j'(x_j)$ and $b = 1 + 2x_j L_j'(x_j)$, which gives

$$H_j(x) = \left[1 - 2(x - x_j) L_j'(x_j) \right] L_j^2(x).$$

For $\widehat{H}_j$, the points x_i with $i \neq j$ must again be double zeros, while x_j must be a simple zero. Hence $\widehat{H}_j(x) = C(x - x_j)L_j^2(x)$. Since $\widehat{H}_j'(x_j) = 1$, we obtain $C = 1$. □

Remark 3.4.2 (Hermite Interpolation Error)**.** If $f \in C^{2n+2}[a, b]$, then for every $x \in [a, b]$ there exists $\xi(x) \in (a, b)$ such that

$$f(x) - H(x) = \frac{f^{(2n+2)}(\xi(x))}{(2n+2)!} \prod_{j=0}^{n} (x - x_j)^2.$$

3.5 Piecewise Polynomial Approximation and Cubic Splines

High-degree polynomial interpolation may oscillate dramatically. Instead of using one high-degree polynomial over the whole interval, a more stable strategy is to divide the interval into smaller subintervals and use low-degree polynomials piecewise.

3.5.1 Piecewise Linear Interpolation

Let $x_0 < x_1 < \cdots < x_n$. The piecewise linear interpolant $I_n f$ satisfies $I_n f \in C[x_0, x_n]$, $I_n f(x_j) = f(x_j)$, and $I_n f$ is linear on every interval $[x_j, x_{j+1}]$. Explicitly,

$$I_n f(x) = f(x_j)\frac{x_{j+1} - x}{x_{j+1} - x_j} + f(x_{j+1})\frac{x - x_j}{x_{j+1} - x_j}, \qquad x \in [x_j, x_{j+1}].$$

Define the hat functions ϕ_i by

$$\phi_i(x) = \begin{cases} \dfrac{x - x_{i-1}}{x_i - x_{i-1}}, & x \in [x_{i-1}, x_i], \\ \dfrac{x_{i+1} - x}{x_{i+1} - x_i}, & x \in [x_i, x_{i+1}], \\ 0, & \text{otherwise}, \end{cases}$$

with the obvious modification at the endpoints. They satisfy $\phi_i(x_j) = \delta_{ij}$, so

$$I_n f(x) = \sum_{i=0}^{n} f(x_i)\phi_i(x).$$

For $x \in [x_j, x_{j+1}]$, the interpolation error is

$$f(x) - I_n f(x) = \frac{1}{2} f''(\xi(x))(x - x_j)(x - x_{j+1})$$

for some $\xi(x) \in (x_j, x_{j+1})$. Therefore, if $h = \max_j (x_{j+1} - x_j)$,

$$\max_{x \in [x_0, x_n]} |f(x) - I_n f(x)| \le \frac{h^2}{8} \max_{x \in [x_0, x_n]} |f''(x)|.$$

3.5.2 Application to a Two-Point Boundary Value Problem

Consider

$$-u''(x) = f(x), \qquad x \in (0,1), \qquad u(0) = u(1) = 0.$$

Its weak formulation is: find $u \in H_0^1(0,1)$ such that

$$(u', v') = (f, v) \qquad \text{for every } v \in H_0^1(0,1).$$

Indeed,

$$\int_0^1 u'v'\,dx = -\int_0^1 u''v\,dx + u'v\big|_0^1 = \int_0^1 fv\,dx,$$

since $v(0) = v(1) = 0$.

Here

$$H_0^1(0,1) = \left\{ v : \int_0^1 |v|^2\,dx < \infty,\ \int_0^1 |v'|^2\,dx < \infty,\ v(0) = v(1) = 0 \right\}.$$

For the piecewise linear finite element method, let $X_n = \operatorname{span}\{\phi_1, \ldots, \phi_{n-1}\}$ and seek

$$u_n(x) = \sum_{j=1}^{n-1} c_j \phi_j(x)$$

such that

$$(u_n', v_n') = (f, v_n) \qquad \text{for every } v_n \in X_n.$$

Taking $v_n = \phi_i$ gives

$$\sum_{j=1}^{n-1} c_j(\phi_j', \phi_i') = (f, \phi_i), \qquad i = 1, \ldots, n-1.$$

Define $m_{ij} = (\phi_j', \phi_i')$ and $f_i = (f, \phi_i)$. Then the coefficient vector C satisfies

$$MC = F.$$

The matrix $M = (m_{ij})$ is symmetric and positive definite, hence invertible. Therefore one can solve for C and reconstruct u_n.

3.5.3 Cubic Splines

A cubic spline is a piecewise cubic interpolant with sufficient smoothness across the interpolation nodes.

Definition 3.5.1. Let $x_0 < x_1 < \cdots < x_n$. A cubic spline interpolant $S(x)$ satisfies:

1. $S_j = S|_{[x_j, x_{j+1}]}$ is a cubic polynomial;
2. $S(x_j) = f(x_j)$ for every interpolation node;
3. $S \in C^2[x_0, x_n]$.

On each interval, write

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3.$$

The interpolation conditions are $S_j(x_j) = f(x_j)$ and $S_j(x_{j+1}) = f(x_{j+1})$. Smoothness additionally requires

$$S_j'(x_{j+1}) = S_{j+1}'(x_{j+1}), \qquad S_j''(x_{j+1}) = S_{j+1}''(x_{j+1}).$$

Two additional boundary conditions are required. Common choices include the *natural* boundary conditions

$$S''(x_0) = S''(x_n) = 0$$

and the *clamped* boundary conditions

$$S'(x_0) = f'(x_0), \qquad S'(x_n) = f'(x_n).$$

Let $h_j = x_{j+1} - x_j$ and define $\mu_j = S''(x_j)$. Since $S_j''(x)$ is linear,

$$S_j''(x) = \mu_j \frac{x_{j+1} - x}{h_j} + \mu_{j+1} \frac{x - x_j}{h_j}.$$

Integrating twice and using $S_j(x_j) = f(x_j)$ and $S_j(x_{j+1}) = f(x_{j+1})$ yields

$$\begin{aligned} S_j(x) = {} & \frac{\mu_j}{6h_j}(x_{j+1} - x)^3 + \frac{\mu_{j+1}}{6h_j}(x - x_j)^3 \\ & + \left(\frac{f(x_j)}{h_j} - \frac{\mu_j h_j}{6} \right)(x_{j+1} - x) \\ & + \left(\frac{f(x_{j+1})}{h_j} - \frac{\mu_{j+1} h_j}{6} \right)(x - x_j). \end{aligned}$$

The condition $S_j'(x_{j+1}) = S_{j+1}'(x_{j+1})$ gives

$$\frac{h_j}{6}\mu_j + \frac{h_j + h_{j+1}}{3}\mu_{j+1} + \frac{h_{j+1}}{6}\mu_{j+2} = f[x_{j+1}, x_{j+2}] - f[x_j, x_{j+1}],$$

or equivalently

$$\frac{h_j}{6}\mu_j + \frac{h_j + h_{j+1}}{3}\mu_{j+1} + \frac{h_{j+1}}{6}\mu_{j+2} = (h_j + h_{j+1})f[x_j, x_{j+1}, x_{j+2}].$$

For the natural cubic spline, $\mu_0 = \mu_n = 0$. Hence the remaining unknowns satisfy a tridiagonal linear system.

Theorem 3.5.2. Let $a = x_0 < x_1 < \cdots < x_n = b$. Then f has a unique natural cubic spline interpolant on the nodes $x_0, x_1, \ldots, x_n$.

Proof. The coefficient matrix of the system for $\mu_1, \ldots, \mu_{n-1}$ is strictly diagonally dominant. Therefore it is invertible, so the linear system has a unique solution. The corresponding natural cubic spline is therefore unique. □

Remark 3.5.3. If $f \in C^4[a, b]$ and $M = \max_{x\in[a,b]} \left|f^{(4)}(x)\right|$, then the cubic spline interpolation error satisfies

$$|f(x) - S(x)| \leq CMh^4,$$

where $h = \max_j h_j$ and C is a constant independent of h. Thus cubic spline interpolation has fourth-order accuracy in the maximum norm under sufficient smoothness.

Chapter 4

Numerical Differentiation and Quadrature

4.1 Finite-Difference Approximations of Derivatives

Numerical differentiation approximates derivatives using function values at nearby discrete points. For a step size $h > 0$, the standard first-order difference operators are

$$D_+ f(x_0) = \frac{f(x_0 + h) - f(x_0)}{h}, \qquad D_- f(x_0) = \frac{f(x_0) - f(x_0 - h)}{h},$$

and the central difference operator is

$$D_0 f(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h}.$$

By Taylor's theorem,

$$f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{h^2}{2} f''(\xi_+),$$

for some $\xi_+ \in (x_0, x_0 + h)$. Therefore

$$D_+ f(x_0) = f'(x_0) + \frac{h}{2} f''(\xi_+),$$

so the forward difference is a first-order approximation:

$$D_+ f(x_0) = f'(x_0) + O(h).$$

Similarly,

$$D_- f(x_0) = f'(x_0) + O(h).$$

For the central difference,

$$f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{h^2}{2} f''(x_0) + \frac{h^3}{6} f^{(3)}(\xi_+),$$

and

$$f(x_0 - h) = f(x_0) - hf'(x_0) + \frac{h^2}{2} f''(x_0) - \frac{h^3}{6} f^{(3)}(\xi_-).$$

Subtracting gives

$$D_0 f(x_0) = f'(x_0) + O(h^2).$$

Hence the central difference is second-order accurate provided that $f \in C^3[x_0 - h, x_0 + h]$.

Definition 4.1.1. A finite-difference approximation $D_h f$ to f' is said to have order p if

$$D_h f(x) - f'(x) = O(h^p)$$

as $h \to 0$.

Thus the forward and backward differences are first-order accurate, while the central difference is second-order accurate.

4.2 General Difference Formulas and Second Derivatives

A general approximation of $f'(x_k)$ may be written as

$$f'(x_k) \approx \sum_{j=0}^{n} a_{kj} f(x_j).$$

One systematic way to determine the coefficients is to interpolate f by a polynomial and differentiate the interpolant.
If

$$f(x) = \sum_{j=0}^{n} f(x_j) L_j(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!} \prod_{j=0}^{n} (x - x_j),$$

then differentiating at x_k gives

$$f'(x_k) = \sum_{j=0}^{n} f(x_j) L_j'(x_k) + \text{error}.$$

Hence $a_{kj} = L_j'(x_k)$.
For two interpolation points x_0, x_1,

$$f(x) = f(x_0)\frac{x - x_1}{x_0 - x_1} + f(x_1)\frac{x - x_0}{x_1 - x_0} + \frac{1}{2} f''(\xi)(x - x_0)(x - x_1).$$

Differentiating gives the forward formula at x_0,

$$f'(x_0) = \frac{f(x_1) - f(x_0)}{x_1 - x_0} + \frac{x_0 - x_1}{2} f''(\xi_0),$$

and the corresponding backward formula at x_1.
With three equally spaced points $x_0, x_0 + h, x_0 + 2h$, one obtains the second-order forward formula

$$f'(x_0) = \frac{-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)}{2h} + O(h^2),$$

and similarly the second-order backward formula

$$f'(x_0) = \frac{3f(x_0) - 4f(x_0 - h) + f(x_0 - 2h)}{2h} + O(h^2).$$

Another way to derive finite-difference formulas is the method of undetermined coefficients.
Suppose, for example, that

$$D_{-,2} f(x_0) = a f(x_0) + b f(x_0 - h) + c f(x_0 - 2h).$$

Expanding around x_0,

$$\begin{aligned} D_{-,2} f(x_0) = {} & (a + b + c) f(x_0) - h(b + 2c) f'(x_0) \\ & + \frac{h^2}{2}(b + 4c) f''(x_0) - \frac{h^3}{6}(b + 8c) f^{(3)}(x_0) + \cdots . \end{aligned}$$

To approximate $f'(x_0)$ with second-order accuracy, require

$$a + b + c = 0, \qquad -h(b + 2c) = 1, \qquad b + 4c = 0.$$

Solving gives

$$a = \frac{3}{2h}, \qquad b = -\frac{2}{h}, \qquad c = \frac{1}{2h},$$

and therefore

$$D_{-,2}f(x_0) = \frac{3f(x_0) - 4f(x_0 - h) + f(x_0 - 2h)}{2h}.$$

For the second derivative, the standard central finite difference is

$$D_2u(x) = \frac{u(x + h) - 2u(x) + u(x - h)}{h^2}.$$

Taylor expansion gives

$$D_2u(x) = u''(x) + \frac{h^2}{12}u^{(4)}(\xi)$$

for some $\xi \in (x - h, x + h)$. Thus $D_2u(x) = u''(x) + O(h^2)$.
In fact,

$$D_2 = D_+D_-.$$

Indeed,

$$D_+D_-u(x) = \frac{1}{h}\left[\frac{u(x + h) - u(x)}{h} - \frac{u(x) - u(x - h)}{h}\right] = D_2u(x).$$

One could also combine noncompact differences such as $D_0D_0u(x)$, but this involves values at $x \pm 2h$. Although it can still be second-order accurate, it uses a wider stencil and is therefore less compact.
For unequal neighboring steps h_1 and h_2, interpolation at x, $x + h_1$, and $x + h_1 + h_2$ yields a second-derivative approximation that is generally only first-order accurate. When $h_1 = h_2 = h$, symmetry cancels the first-order term and recovers the second-order central difference.

4.3 Round-Off Error and Finite-Difference Discretization

Finite-difference formulas involve division by powers of h. Therefore, although the truncation error decreases as $h \to 0$, round-off error may increase.
For the midpoint formula, suppose the computed values satisfy

$$\widetilde{f}(x_0 + h) = f(x_0 + h) + e_+, \qquad \widetilde{f}(x_0 - h) = f(x_0 - h) + e_-,$$

with $|e_\pm| \le \varepsilon$. Then

$$\widetilde{D}_0f(x_0) = \frac{\widetilde{f}(x_0 + h) - \widetilde{f}(x_0 - h)}{2h}.$$

The total error satisfies

$$\left|f'(x_0) - \widetilde{D}_0f(x_0)\right| \le C_1h^2 + \frac{\varepsilon}{h},$$

where C_1 depends on a bound for $f^{(3)}$.
Thus choosing h extremely small does not necessarily improve the approximation. The first term decreases as $h \to 0$, while the second increases. Balancing the two terms gives an optimal scale of approximately $h \sim \varepsilon^{1/3}$.

4.3.1 From a Continuous Boundary Value Problem to a Discrete System

Consider the two-point boundary value problem

$$-u''(x) = f(x), \qquad x \in (0,1), \qquad u(0) = u(1) = 0.$$

Set $x_j = jh$, where $h = 1/(n+1)$, and let $U_j \approx u(x_j)$, $f_j = f(x_j)$. The boundary values are $U_0 = U_{n+1} = 0$.
Replacing $u''(x_j)$ by the central finite difference gives

$$-\frac{U_{j-1} - 2U_j + U_{j+1}}{h^2} = f_j, \qquad j = 1, \ldots, n.$$

Equivalently,

$$AU = F,$$

where

$$A = \frac{1}{h^2}\begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}, \qquad U = \begin{pmatrix} U_1 \\ U_2 \\ \vdots \\ U_n \end{pmatrix}, \qquad F = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix}.$$

The matrix A is the discrete differentiation matrix: it maps discrete function values to approximations of the second derivative.

Definition 4.3.1. For a differential operator L and its discrete approximation L_h, the *local truncation error* at a grid point is

$$\tau_j = L_h u(x_j) - Lu(x_j),$$

where the exact solution u is substituted into the discrete formula.

For the central second difference, $\tau_j = O(h^2)$, so the discretization is second-order consistent.

Definition 4.3.2. A discretization is *consistent* if its truncation error tends to zero as $h \to 0$. It is *stable* if perturbations in the discrete equations produce controlled perturbations in the discrete solution. It is *convergent* if the discrete solution approaches the exact solution as the mesh is refined.

The basic philosophy is

$$\boxed{\text{consistency} + \text{stability} \Longrightarrow \text{convergence}}$$

under suitable assumptions. The truncation error measures the quality of the local discretization, whereas the global error measures the accumulated difference between U_j and $u(x_j)$.

4.4 Interpolatory Quadrature and Newton–Cotes Formulas

Numerical integration seeks to approximate

$$\int_a^b f(x)\,dx$$

using finitely many function values. This is necessary when an integral cannot be evaluated explicitly or when f is available only through discrete data $\{(x_j, f(x_j))\}$.
Suppose

$$f(x) = \sum_{j=0}^{n} f(x_j)L_j(x) + R_n(x).$$

Integrating gives

$$\int_a^b f(x)\,dx = \sum_{j=0}^{n} f(x_j)\int_a^b L_j(x)\,dx + \int_a^b R_n(x)\,dx.$$

Hence an interpolatory quadrature rule has the form

$$I_n(f) = \sum_{j=0}^{n} a_j f(x_j), \qquad a_j = \int_a^b L_j(x)\,dx.$$

For equally spaced nodes, the resulting formulas are called *Newton–Cotes formulas*.

4.4.1 Midpoint, Trapezoidal, and Simpson Rules

The midpoint rule is

$$\int_a^b f(x)\,dx = (b-a)f\left(\frac{a+b}{2}\right) + \frac{(b-a)^3}{24}f''(\xi)$$

for some $\xi \in (a,b)$.
For $n = 1$, interpolation at a and b gives the trapezoidal rule

$$\int_a^b f(x)\,dx = \frac{b-a}{2}\big(f(a)+f(b)\big) - \frac{(b-a)^3}{12}f''(\xi).$$

For $n = 2$, using the nodes $a, (a+b)/2, b$, one obtains Simpson's rule

$$\int_a^b f(x)\,dx = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right] - \frac{(b-a)^5}{2880}f^{(4)}(\xi).$$

Definition 4.4.1. The *degree of precision* of a quadrature formula is the largest integer m such that the formula is exact for every polynomial of degree at most m.

The trapezoidal rule has degree of precision 1, while Simpson's rule has degree of precision 3.
For a general closed Newton–Cotes formula, take $x_i = a + ih$, $i = 0, \dots, n$, with $h = (b-a)/n$. Then

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n} a_i f(x_i), \qquad a_i = \int_a^b L_i(x)\,dx.$$

The interpolation remainder shows that the degree of precision is at least n. For certain values of n, symmetry increases the degree by one.

Examples include

$$\int_{x_0}^{x_1} f(x)\,dx \approx \frac{h}{2}\Big(f(x_0)+f(x_1)\Big),$$

$$\int_{x_0}^{x_2} f(x)\,dx \approx \frac{h}{3}\Big(f(x_0)+4f(x_1)+f(x_2)\Big),$$

and Simpson's three-eighth rule

$$\int_{x_0}^{x_3} f(x)\,dx \approx \frac{3h}{8}\Big(f(x_0)+3f(x_1)+3f(x_2)+f(x_3)\Big).$$

Open Newton–Cotes formulas use interpolation points strictly inside the interval rather than including the endpoints.

4.5 Composite Quadrature and Round-Off Stability

High-degree Newton–Cotes formulas have two disadvantages. Their weights may grow rapidly, and some weights become negative. This can make the formula sensitive to round-off error.

If

$$I_n(f) = \sum_{i=0}^{n} a_i f(x_i)$$

and the computed data satisfy $\left|f(x_i) - \widetilde{f}(x_i)\right| \le \varepsilon$, then

$$\left|I_n(f) - I_n(\widetilde{f})\right| \le \varepsilon \sum_{i=0}^{n} |a_i|\,.$$

If all weights are nonnegative, then

$$\sum_{i=0}^{n} |a_i| = \sum_{i=0}^{n} a_i = b - a,$$

because the formula is exact for the constant function 1. Hence

$$\left|I_n(f) - I_n(\widetilde{f})\right| \le (b-a)\varepsilon.$$

Therefore a quadrature formula with nonnegative weights is stable with respect to round-off error.

For high-order Newton–Cotes rules, negative weights may appear; in particular, sufficiently high degree can lead to poor numerical stability.

A preferable strategy for a large interval is to divide it into many small subintervals and apply a low-order quadrature formula piecewise.

Let $x_j = a + jh$, $j = 0, \ldots, n$, with $h = (b-a)/n$.

The composite trapezoidal rule is

$$\int_a^b f(x)\,dx \approx \frac{h}{2}\left[f(x_0) + 2\sum_{j=1}^{n-1} f(x_j) + f(x_n)\right].$$

Its error has order $O(h^2)$.

For even $n = 2m$, the composite Simpson rule is

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\Bigg[f(x_0) + 4\sum_{j=1}^{m} f(x_{2j-1}) + 2\sum_{j=1}^{m-1} f(x_{2j}) + f(x_{2m})\Bigg].$$

Its error has order $O(h^4)$.
Thus piecewise low-order approximation is often preferable to using one very high-degree interpolating polynomial over the whole interval.

4.6 Gaussian Quadrature and Further Applications

Newton–Cotes formulas prescribe the interpolation nodes in advance. Gaussian quadrature instead chooses both the nodes and weights in order to maximize the degree of precision.
Consider

$$Q(f) = \sum_{j=0}^{n} a_j f(x_j).$$

Definition 4.6.1. If a quadrature formula using $n+1$ nodes has degree of precision $2n+1$, then the nodes x_j are called *Gaussian points*, and the formula is called a *Gaussian quadrature formula.*

Let

$$\omega_{n+1}(x) = \prod_{j=0}^{n}(x - x_j).$$

A necessary and sufficient condition for a Gaussian rule is that

$$\int_a^b p(x)\omega_{n+1}(x)\,dx = 0$$

for every polynomial p of degree at most n.
Indeed, suppose first that the rule has degree $2n+1$. For $p \in \mathcal{P}_n$, the polynomial $p\omega_{n+1}$ has degree at most $2n+1$, so

$$\int_a^b p(x)\omega_{n+1}(x)\,dx = \sum_{j=0}^{n} a_j p(x_j)\omega_{n+1}(x_j) = 0.$$

Conversely, let $f \in \mathcal{P}_{2n+1}$. By polynomial division,

$$f(x) = p(x)\omega_{n+1}(x) + q(x),$$

where $p, q \in \mathcal{P}_n$. Orthogonality gives

$$\int_a^b f(x)\,dx = \int_a^b q(x)\,dx.$$

Since $\omega_{n+1}(x_j) = 0$, $f(x_j) = q(x_j)$, and therefore

$$\int_a^b f(x)\,dx = \sum_{j=0}^{n} a_j f(x_j).$$

Thus the degree of precision is $2n+1$.

4.6.1 Legendre Polynomials and Gauss–Legendre Quadrature

On $[-1,1]$, Gaussian points are obtained from Legendre polynomials.

Definition 4.6.2. The Legendre polynomial $P_n(x)$ is a polynomial of degree n that is orthogonal to every polynomial of lower degree:

$$\int_{-1}^{1} P_n(x)q(x)\,dx = 0 \qquad \text{for every } \deg q < n.$$

Up to normalization,

$$P_0(x) = 1, \qquad P_1(x) = x, \qquad P_2(x) = x^2 - \frac{1}{3},$$

and higher-degree Legendre polynomials can be generated recursively.
The roots of the $(n+1)$-st Legendre polynomial are the Gaussian nodes for a rule with $n+1$ points.
For example, the two-point Gauss–Legendre rule uses $x_1 = -1/\sqrt{3}$ and $x_2 = 1/\sqrt{3}$:

$$\int_{-1}^{1} f(x)\,dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

It is exact for every polynomial of degree at most 3.
A quadrature rule on a general interval $[a,b]$ can be transformed to $[-1,1]$ using

$$x = \frac{b-a}{2}t + \frac{a+b}{2}.$$

4.6.2 Multiple Integrals

For a rectangular domain $[a,b] \times [c,d]$, one may apply a one-dimensional quadrature formula successively in each coordinate.
For example, applying Simpson's rule in both variables gives a tensor-product formula. If $x_m = (a+b)/2$ and $y_m = (c+d)/2$, then

$$\begin{aligned}\int_a^b \int_c^d f(x,y)\,dy\,dx \approx \frac{(b-a)(d-c)}{36}\Big[&f(a,c) + 4f(x_m,c) + f(b,c)\\ &+ 4f(a,y_m) + 16f(x_m,y_m) + 4f(b,y_m)\\ &+ f(a,d) + 4f(x_m,d) + f(b,d)\Big].\end{aligned}$$

For a nonrectangular domain

$$D = \{(x,y) : a \le x \le b,\ c(x) \le y \le d(x)\},$$

one can first apply quadrature in y and then approximate the resulting one-dimensional integral in x.

4.6.3 Improper Integrals

If an integrand has an endpoint singularity, direct application of standard quadrature rules may be inaccurate.
For example, consider

$$I = \int_a^b \frac{g(x)}{(x-a)^p}\,dx, \qquad 0 < p < 1.$$

Expand g near a:

$$g(x) = g(a) + g'(a)(x-a) + \frac{g''(a)}{2}(x-a)^2 + \cdots .$$

Separating several terms of this expansion gives

$$I = \int_a^b \frac{P_m(x)}{(x-a)^p}\,dx + \int_a^b \frac{g(x) - P_m(x)}{(x-a)^p}\,dx.$$

The first integral can often be evaluated analytically, while the second has a weaker singularity and can be treated numerically.
Another useful strategy is a change of variables chosen specifically to remove the singular behavior. For infinite intervals, a transformation can similarly map the original domain to a finite interval before applying a standard quadrature method.

Chapter 5

Numerical Methods for Ordinary Differential Equations

5.1 Initial Value Problems and the Forward Euler Method

Consider the initial value problem

$$\begin{cases} y'(t) = f(t, y(t)), & 0 < t < T, \\ y(0) = y_0, \end{cases}$$

where f is Lipschitz continuous in its second variable.
Let $t_n = nh$, $n = 0, 1, \ldots, N$, where $h = T/N$. We approximate $y(t)$ only at the discrete points $t_0, t_1, \ldots, t_N$. Denote the numerical approximation by $Y_n \approx y(t_n)$, and define the global error by $e_n = y(t_n) - Y_n$.
By Taylor expansion,

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2} y''(\xi_n), \qquad \xi_n \in (t_n, t_{n+1}).$$

Since $y'(t_n) = f(t_n, y(t_n))$,

$$y(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)) + \tau_{n+1},$$

where

$$\tau_{n+1} = \frac{h^2}{2} y''(\xi_n).$$

Ignoring the truncation term gives the *Forward Euler method*

$$\boxed{Y_{n+1} = Y_n + hf(t_n, Y_n)}.$$

Definition 5.1.1. For a one-step method, the *local truncation error* is the error produced in one step when the exact solution is substituted into the numerical formula.

For Forward Euler, $\tau_{n+1} = O(h^2)$. Thus the local truncation error is second order in h, even though the accumulated global error will be only first order.

5.2 Global Error Analysis of the Euler Method

Assume that f is Lipschitz continuous in y with Lipschitz constant L:

$$|f(t, y_1) - f(t, y_2)| \le L\,|y_1 - y_2|\,.$$

Let $M = \max_{0 \le t \le T} |y''(t)|$. Then $|\tau_{n+1}| \le Mh^2/2$.
The exact and numerical solutions satisfy

$$y(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)) + \tau_{n+1}$$

and

$$Y_{n+1} = Y_n + hf(t_n, Y_n).$$

Subtracting gives

$$e_{n+1} = e_n + h\Big[f(t_n, y(t_n)) - f(t_n, Y_n)\Big] + \tau_{n+1}.$$

Hence

$$|e_{n+1}| \le (1 + hL)\,|e_n| + \frac{1}{2} Mh^2.$$

We use two elementary inequalities.

Lemma 5.2.1. For $x > -1$ and $m > 0$,

$$0 \le (1+x)^m \le e^{mx}.$$

Lemma 5.2.2. Suppose $s > 0$, $t > 0$, and a nonnegative sequence $\{a_i\}$ satisfies

$$a_{i+1} \le (1+s)a_i + t.$$

Then

$$a_{i+1} \le (1+s)^{i+1}a_0 + \frac{(1+s)^{i+1}-1}{s}\,t.$$

Proof. Repeated substitution gives

$$\begin{aligned} a_{i+1} &\le (1+s)^{i+1}a_0 + \left[1 + (1+s) + \cdots + (1+s)^i\right] t \\ &= (1+s)^{i+1}a_0 + \frac{(1+s)^{i+1}-1}{s}\,t. \end{aligned}$$

□

Applying the lemma with $s = hL$ and $t = Mh^2/2$,

$$|e_n| \le (1+hL)^n\,|e_0| + \frac{Mh}{2L}\left[(1+hL)^n - 1\right].$$

Since $(1+hL)^n \le e^{nhL} = e^{t_n L}$,

$$\boxed{|e_n| \le e^{t_n L}\,|e_0| + \frac{M}{2L}\left(e^{t_n L} - 1\right) h}.$$

If the initial value is exact, $e_0 = 0$, then

$$|e_n| \le \frac{M}{2L}\left(e^{t_n L} - 1\right) h = O(h).$$

Therefore Forward Euler is a first-order method in the global error.

Remark 5.2.3. The local truncation error of Forward Euler is $O(h^2)$, while the global error is $O(h)$. Roughly speaking, there are $O(h^{-1})$ time steps over a fixed interval, so local errors accumulate over the entire computation.

Remark 5.2.4. If $t_n L$ is large, the factor $e^{t_n L}$ can amplify errors substantially. Thus a smaller step size may be required over long time intervals or for problems with a large Lipschitz constant.

5.2.1 Effect of Round-Off Error

In an actual computation, the Forward Euler step may be written as

$$\widetilde{Y}_{n+1} = \widetilde{Y}_n + hf(t_n, \widetilde{Y}_n) + \delta_n,$$

where $|\delta_n| \le \varepsilon$ represents round-off error.
The error recurrence becomes

$$|e_{n+1}| \le (1+hL)\,|e_n| + \frac{1}{2}Mh^2 + \varepsilon.$$

Consequently, the accumulated error contains both a truncation contribution of order h and a round-off contribution that behaves roughly like ε/h. Thus taking h arbitrarily small is not always numerically optimal.

5.3 Explicit and Implicit One-Step Methods

The differential equation can also be written in integral form:

$$y(t_k) = y(t_{k-1}) + \int_{t_{k-1}}^{t_k} f(t, y(t))\, dt.$$

Different numerical methods result from different approximations of this integral.

5.3.1 Forward Euler Method

Approximating $f(t, y(t))$ by $f(t_{k-1}, Y_{k-1})$ gives

$$Y_k = Y_{k-1} + hf(t_{k-1}, Y_{k-1}).$$

This method is explicit, since Y_k is determined directly from previously known quantities. Its local truncation error is $O(h^2)$.

5.3.2 Backward Euler Method

Using the right endpoint gives

$$\boxed{Y_k = Y_{k-1} + hf(t_k, Y_k)}.$$

The unknown Y_k appears on both sides, so Backward Euler is an implicit method. Its local truncation error is also $O(h^2)$.

5.3.3 Trapezoidal Rule and Crank–Nicolson Scheme

Applying the trapezoidal rule to the integral gives

$$\boxed{Y_k = Y_{k-1} + \frac{h}{2}\left[f(t_{k-1}, Y_{k-1}) + f(t_k, Y_k)\right]}.$$

This is an implicit method. Its local truncation error is $O(h^3)$, so its global error is second order.

To see the global order, suppose the local residual R_k satisfies $|R_k| \le Ch^3$. Then

$$|e_k| \le |e_{k-1}| + \frac{Lh}{2}\left(|e_{k-1}| + |e_k|\right) + Ch^3.$$

If $Lh < 1$, then

$$\left(1 - \frac{Lh}{2}\right)|e_k| \le \left(1 + \frac{Lh}{2}\right)|e_{k-1}| + Ch^3.$$

Hence

$$|e_k| \le (1 + 2Lh)\,|e_{k-1}| + 2Ch^3$$

for sufficiently small h, and repeated application gives $|e_k| = O(h^2)$.

5.3.4 Modified Euler Method

A predictor value is first obtained by Forward Euler:

$$Y_k^* = Y_{k-1} + hf(t_{k-1}, Y_{k-1}).$$

The slope is then corrected using the average of the initial and predicted slopes:

$$\boxed{Y_k = Y_{k-1} + \frac{h}{2}\left[f(t_{k-1}, Y_{k-1}) + f(t_k, Y_k^*)\right]}.$$

Equivalently,

$$Y_k = Y_{k-1} + \frac{h}{2}\left[f(t_{k-1}, Y_{k-1}) + f(t_k, Y_{k-1} + hf(t_{k-1}, Y_{k-1}))\right].$$

The local truncation error is $O(h^3)$, so the method is second-order accurate globally.

5.3.5 Midpoint Method

The midpoint method uses an approximation to the solution halfway through the step:

$$\boxed{Y_k = Y_{k-1} + hf\left(t_{k-1} + \frac{h}{2}, Y_{k-1} + \frac{h}{2} f(t_{k-1}, Y_{k-1})\right)}.$$

Its local truncation error is $O(h^3)$, and therefore its global order is two.

5.4 Runge–Kutta Methods

The main idea of a Runge–Kutta method is to approximate

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt$$

by combining several evaluations of f within one time step.
A general explicit r-stage Runge–Kutta method has the form

$$Y_{n+1} = Y_n + h \sum_{i=1}^{r} c_i k_i,$$

where

$$k_1 = f(t_n, Y_n)$$

and, for $i = 2, \ldots, r$,

$$k_i = f\left(t_n + \lambda_i h, Y_n + h \sum_{j=1}^{i-1} \mu_{ij} k_j\right).$$

The coefficients c_i, λ_i, and μ_{ij} are chosen so that the Taylor expansion of the numerical method agrees with the Taylor expansion of the exact solution to as high an order as possible.

5.4.1 Second-Order Runge–Kutta Construction

For a two-stage method, consider

$$Y_{n+1} = Y_n + h\left[a_1 f(t_n, Y_n) + a_2 f(t_n + \alpha h, Y_n + \beta h f(t_n, Y_n))\right].$$

Taylor expansion gives

$$f(t + \alpha h, y + \beta h f) = f(t, y) + \alpha h f_t(t, y) + \beta h f_y(t, y) f(t, y) + O(h^2).$$

Matching this expansion with the Taylor expansion of the exact solution produces

$$a_1 + a_2 = 1, \qquad a_2 \alpha = \frac{1}{2}, \qquad a_2 \beta = \frac{1}{2}.$$

Different choices satisfying these conditions give different second-order Runge–Kutta methods. For example, taking $a_1 = a_2 = 1/2$ and $\alpha = \beta = 1$ gives the modified Euler method, while choosing $a_1 = 0$, $a_2 = 1$, and $\alpha = \beta = 1/2$ gives the midpoint method.

5.4.2 Classical Fourth-Order Runge–Kutta Method

The classical fourth-order Runge–Kutta method is

$$\begin{aligned} k_1 &= hf(t_i, W_i), \\ k_2 &= hf\left(t_i + \frac{h}{2}, W_i + \frac{k_1}{2}\right), \\ k_3 &= hf\left(t_i + \frac{h}{2}, W_i + \frac{k_2}{2}\right), \\ k_4 &= hf(t_i + h, W_i + k_3), \end{aligned}$$

followed by

$$\boxed{W_{i+1} = W_i + \frac{1}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)}.$$

The method combines four slope evaluations and has fourth-order global accuracy for sufficiently smooth solutions.

5.5 Consistency, Stability, and Convergence

The error analysis above can be organized using three general concepts.

Definition 5.5.1. A numerical method is *consistent* if its local truncation error tends to zero as $h \to 0$.

Definition 5.5.2. A numerical method is *stable* if small perturbations in the initial data, intermediate numerical values, or local errors do not produce uncontrolled growth in the numerical solution.

Definition 5.5.3. A numerical method is *convergent of order* p if, for fixed T,

$$\max_{0 \le n \le N} |y(t_n) - Y_n| = O(h^p)$$

as $h \to 0$.

For the methods considered in this chapter:

Method	Type	Local Error	Global Order
Forward Euler	Explicit	$O(h^2)$	1
Backward Euler	Implicit	$O(h^2)$	1
Trapezoidal / Crank–Nicolson	Implicit	$O(h^3)$	2
Modified Euler	Explicit	$O(h^3)$	2
Midpoint	Explicit	$O(h^3)$	2
Classical RK4	Explicit	$O(h^5)$	4

The central principle is that a small local truncation error alone is not sufficient: the errors introduced at individual steps must also remain controlled as they propagate through the computation. For stable one-step methods, consistency leads to convergence, and the global order is typically one power of h lower than the local truncation error order.

Chapter 6

Direct Methods for Linear Systems

6.1 Gaussian Elimination and Backward Substitution

Consider the linear system

$$A\boldsymbol{x} = \boldsymbol{b},$$

where $A \in \mathbb{R}^{n\times n}$. Gaussian elimination transforms the augmented matrix $[A \mid \boldsymbol{b}]$ into an upper triangular system by elementary row operations.
Let E_k denote the k-th row. At the first elimination step, provided $a_{11} \neq 0$, perform

$$E_j \leftarrow E_j - \frac{a_{j1}}{a_{11}} E_1, \qquad j = 2, \ldots, n.$$

The entries below the first pivot are then zero.
At the second step, provided the new pivot $a_{22}^{(2)} \neq 0$, perform

$$E_j \leftarrow E_j - \frac{a_{j2}^{(2)}}{a_{22}^{(2)}} E_2, \qquad j = 3, \ldots, n.$$

Continuing in this way, at step k use the multiplier

$$m_{jk} = \frac{a_{jk}^{(k)}}{a_{kk}^{(k)}}, \qquad j = k+1, \ldots, n,$$

and perform $E_j \leftarrow E_j - m_{jk} E_k$.
After $n-1$ elimination steps, the system has the form

$$U\boldsymbol{x} = \boldsymbol{c},$$

where U is upper triangular. The solution is obtained by backward substitution:

$$x_n = \frac{c_n}{u_{nn}},$$

and, for $i = n-1, \ldots, 1$,

$$x_i = \frac{1}{u_{ii}} \left(c_i - \sum_{j=i+1}^{n} u_{ij} x_j \right).$$

If a pivot $a_{kk}^{(k)} = 0$, but there exists $p > k$ such that $a_{pk}^{(k)} \neq 0$, exchange rows E_k and E_p. This does not change the solution set.
If no such row exists, then A is singular, and the system may have no solution or infinitely many solutions.

6.2 Pivoting and Round-Off Stability

Even when a pivot is nonzero, a very small pivot can cause severe round-off error because the multiplier m_{jk} becomes large.
A practical remedy is *partial pivoting*. At step k, choose $p \in \{k, k+1, \ldots, n\}$ such that

$$\left| a_{pk}^{(k)} \right| = \max_{k \le i \le n} \left| a_{ik}^{(k)} \right|,$$

exchange E_k and E_p, and then continue Gaussian elimination.

Remark 6.2.1. The purpose of pivoting is not only to avoid division by zero. It also reduces the amplification of round-off errors caused by division by very small pivot elements.

Example 6.2.2. With four-digit rounding, consider

$$0.0030x_1 + 59.14x_2 = 59.17,$$

$$5.291x_1 - 6.130x_2 = 46.78.$$

If the first equation is used as the initial pivot row, the multiplier is very large, and substantial loss of accuracy occurs.
Instead, partial pivoting exchanges the two equations so that 5.291 is used as the first pivot. The elimination then proceeds with a much smaller multiplier and produces a substantially more accurate result.

6.3 Operation Counts and LU Decomposition

The computational cost of Gaussian elimination is dominated by the elimination phase. At step k, approximately $(n-k)(n-k+1)$ arithmetic operations are required. Summing over $k = 1, \ldots, n-1$ gives a total leading cost of order $O(n^3)$.
Backward substitution requires only $O(n^2)$ operations. Hence the total cost of solving a dense linear system by Gaussian elimination is $O(n^3)$.
The elimination process can be interpreted as a matrix factorization.
At the first step, let

$$M^{(1)} = \begin{pmatrix} 1 & & & \\ -m_{21} & 1 & & \\ \vdots & & \ddots & \\ -m_{n1} & & & 1 \end{pmatrix}.$$

Then

$$A^{(2)} = M^{(1)}A.$$

At the next step,

$$A^{(3)} = M^{(2)}A^{(2)},$$

and continuing gives

$$U = M^{(n-1)} \cdots M^{(2)}M^{(1)}A.$$

Therefore

$$A = \left(M^{(1)}\right)^{-1} \left(M^{(2)}\right)^{-1} \cdots \left(M^{(n-1)}\right)^{-1} U.$$

The product of the inverse elimination matrices is lower triangular. Hence

$$\boxed{A = LU},$$

where L is lower triangular and U is upper triangular.
For example,

$$L = \begin{pmatrix} 1 & & & \\ m_{21} & 1 & & \\ \vdots & \vdots & \ddots & \\ m_{n1} & m_{n2} & \cdots & 1 \end{pmatrix}.$$

Once $A = LU$ is known, solving $A\boldsymbol{x} = \boldsymbol{b}$ is equivalent to

$$L\boldsymbol{y} = \boldsymbol{b}, \qquad U\boldsymbol{x} = \boldsymbol{y}.$$

The first system is solved by forward substitution and the second by backward substitution, each requiring $O(n^2)$ operations.

Remark 6.3.1. Computing the LU factorization itself still requires $O(n^3)$ operations. However, if several systems

$$A\boldsymbol{x} = \boldsymbol{b}_1, \quad A\boldsymbol{x} = \boldsymbol{b}_2, \quad \dots$$

must be solved with the same matrix A, LU decomposition is especially efficient because the factorization is computed only once. Each additional right-hand side then requires only $O(n^2)$ work.

6.4 Permutation Matrices and Pivoted LU Factorization

A permutation matrix represents a reordering of rows or columns.
Let $k_1, k_2, \dots, k_n$ be a permutation of $1, 2, \dots, n$. The corresponding permutation matrix $P = (p_{ij})$ is defined by

$$p_{ij} = \begin{cases} 1, & j = k_i, \\ 0, & \text{otherwise.} \end{cases}$$

Multiplying a vector by P reorders its components. Multiplying a matrix from the left by P performs the corresponding row exchanges.
Permutation matrices satisfy

$$P^{-1} = P^T, \qquad PP^T = P^T P = I.$$

If Gaussian elimination requires row exchanges, then the permuted system is

$$PA\boldsymbol{x} = P\boldsymbol{b}.$$

Under suitable nonsingularity assumptions, there exists a permutation matrix P such that

$$\boxed{PA = LU}.$$

Equivalently,

$$A = P^T LU.$$

Thus partial pivoting can be represented algebraically by a permutation matrix followed by an LU factorization.

6.5 Special Matrix Structures

Certain matrix structures guarantee nonsingularity or permit more stable and efficient direct solvers.

6.5.1 Strictly Diagonally Dominant Matrices

Definition 6.5.1. A matrix $A = (a_{ij})$ is *strictly diagonally dominant* if

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

for every i.

Theorem 6.5.2. Every strictly diagonally dominant matrix is nonsingular.

Proof. Assume $A\boldsymbol{x} = 0$ for some nonzero $\boldsymbol{x}$. Choose k such that $|x_k| = \max_j |x_j| > 0$. From the k-th equation,

$$a_{kk}x_k = -\sum_{j\neq k} a_{kj}x_j.$$

Therefore

$$|a_{kk}|\,|x_k| \leq \sum_{j\neq k} |a_{kj}|\,|x_j| \leq \left(\sum_{j\neq k} |a_{kj}|\right)|x_k|\,.$$

Since $|x_k| > 0$,

$$|a_{kk}| \leq \sum_{j\neq k} |a_{kj}|\,,$$

contradicting strict diagonal dominance. Hence the only solution of $A\boldsymbol{x} = 0$ is $\boldsymbol{x} = 0$, so A is nonsingular. □

For an SDD matrix, Gaussian elimination can be performed without row interchanges, and the elimination retains favorable numerical stability with respect to round-off errors.

6.5.2 Symmetric Positive Definite Matrices

Definition 6.5.3. A real matrix A is *symmetric positive definite* if $A = A^T$ and

$$\boldsymbol{x}^T A\boldsymbol{x} > 0$$

for every nonzero $\boldsymbol{x} \in \mathbb{R}^n$.

For an SPD matrix, Gaussian elimination can be performed without row interchanges, and all pivot elements are positive.
An SPD matrix also admits an LDL^T factorization,

$$A = LDL^T,$$

where L is lower triangular and D is diagonal with positive entries.
Since the diagonal entries of D are positive, write $D = D^{1/2}D^{1/2}$. Then

$$A = LD^{1/2}\left(LD^{1/2}\right)^T.$$

This yields the Cholesky factorization.

Theorem 6.5.4 (Cholesky Factorization)**.** If A is symmetric positive definite, then there exists a lower triangular matrix L with positive diagonal entries such that

$$\boxed{A = LL^T}.$$

The entries of $L = (l_{ij})$ can be computed recursively. First, $l_{11} = \sqrt{a_{11}}$. For $j = 2, \ldots, n$,

$$l_{j1} = \frac{a_{j1}}{l_{11}}.$$

For $i = 2, \ldots, n$,

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}.$$

For $j = i+1, \ldots, n$,

$$l_{ji} = \frac{a_{ji} - \sum_{k=1}^{i-1} l_{jk}l_{ik}}{l_{ii}}.$$

Cholesky factorization exploits symmetry and therefore requires less storage and fewer operations than a general LU factorization.

6.5.3 Tridiagonal Matrices

Consider a tridiagonal matrix

$$A = \begin{pmatrix} \alpha_1 & \gamma_1 & & & \\ \beta_2 & \alpha_2 & \gamma_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \beta_{n-1} & \alpha_{n-1} & \gamma_{n-1} \\ & & & \beta_n & \alpha_n \end{pmatrix}.$$

Its LU factors have the form

$$L = \begin{pmatrix} 1 & & & \\ l_2 & 1 & & \\ & \ddots & \ddots & \\ & & l_n & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} u_1 & w_1 & & \\ & u_2 & w_2 & \\ & & \ddots & \ddots \\ & & & u_n \end{pmatrix}.$$

Set $u_1 = \alpha_1$ and $w_1 = \gamma_1$. For $j = 2, \ldots, n$,

$$l_j = \frac{\beta_j}{u_{j-1}}, \qquad u_j = \alpha_j - l_j w_{j-1},$$

and, for $j < n$, $w_j = \gamma_j$.
Because only three diagonals are involved, the factorization requires only $O(n)$ operations. Forward and backward substitution also each require $O(n)$ operations.

Remark 6.5.5. This linear complexity is a major advantage in problems produced by one-dimensional finite-difference discretizations, where tridiagonal systems occur naturally.

Chapter 7

Iterative Methods for Linear Systems

7.1 Vector Norms and Matrix Norms

Definition 7.1.1. A *vector norm* on $\mathbb{R}^n$ is a function $\|\cdot\| : \mathbb{R}^n \to [0, \infty)$ satisfying, for all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$,

1. $\|\boldsymbol{x}\| \geq 0$;
2. $\|\boldsymbol{x}\| = 0$ if and only if $\boldsymbol{x} = 0$;
3. $\|\alpha\boldsymbol{x}\| = |\alpha| \, \|\boldsymbol{x}\|$;
4. $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$.

Important examples are the p-norms

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}, \qquad 1 \leq p < \infty,$$

and the maximum norm

$$\|\boldsymbol{x}\|_\infty = \max_{1 \leq i \leq n} |x_i| \, .$$

In particular,

$$\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i| \, , \qquad \|\boldsymbol{x}\|_2 = \left(\sum_{i=1}^{n} x_i^2 \right)^{1/2} .$$

The Euclidean norm satisfies the Cauchy–Schwarz inequality

$$\left| \boldsymbol{x}^T \boldsymbol{y} \right| \leq \|\boldsymbol{x}\|_2 \, \|\boldsymbol{y}\|_2 \, .$$

More generally, for a function f on $[a, b]$, the corresponding L^p-norm is

$$\|f\|_{L^p} = \left(\int_a^b |f(x)|^p \, dx \right)^{1/p} .$$

Theorem 7.1.2. All norms on a finite-dimensional vector space are equivalent. In particular, for any two norms $\|\cdot\|_a$ and $\|\cdot\|_b$ on $\mathbb{R}^n$, there exist constants $C_1, C_2 > 0$ such that

$$C_1 \|\boldsymbol{x}\|_a \leq \|\boldsymbol{x}\|_b \leq C_2 \|\boldsymbol{x}\|_a$$

for every $\boldsymbol{x} \in \mathbb{R}^n$.

For example,

$$\|\boldsymbol{x}\|_\infty \leq \|\boldsymbol{x}\|_2 \leq \sqrt{n} \, \|\boldsymbol{x}\|_\infty \, .$$

Consequently, convergence of a sequence in $\mathbb{R}^n$ is independent of the particular norm chosen.

Definition 7.1.3. A sequence $\{\boldsymbol{x}^{(k)}\}$ converges to $\boldsymbol{x}$ with respect to a norm $\|\cdot\|$ if for every $\varepsilon > 0$, there exists N such that $\left\| \boldsymbol{x}^{(k)} - \boldsymbol{x} \right\| < \varepsilon$ whenever $k \geq N$.

Because all norms in finite dimensions are equivalent, $\boldsymbol{x}^{(k)} \to \boldsymbol{x}$ if and only if every component satisfies $x_j^{(k)} \to x_j$.

Definition 7.1.4. A *matrix norm* is a function on $\mathbb{R}^{m \times n}$ satisfying positivity, definiteness, homogeneity, and the triangle inequality. For square matrices, we additionally require

$$\|AB\| \leq \|A\| \, \|B\| \, .$$

If $\|\cdot\|$ is a vector norm, the corresponding induced matrix norm is

$$\|A\| = \max_{\boldsymbol{x}\neq 0} \frac{\|A\boldsymbol{x}\|}{\|\boldsymbol{x}\|} = \max_{\|\boldsymbol{x}\|=1} \|A\boldsymbol{x}\| .$$

Therefore

$$\|A\boldsymbol{x}\| \leq \|A\| \, \|\boldsymbol{x}\| .$$

For the induced 1-norm,

$$\|A\|_1 = \max_j \sum_i |a_{ij}| ,$$

the maximum absolute column sum. For the induced infinity norm,

$$\|A\|_\infty = \max_i \sum_j |a_{ij}| ,$$

the maximum absolute row sum.
For the Euclidean norm,

$$\|A\|_2 = \sqrt{\rho(A^T A)},$$

where ρ denotes the spectral radius.

Remark 7.1.5. The Frobenius norm

$$\|A\|_F = \left(\sum_{i,j} |a_{ij}|^2 \right)^{1/2}$$

is a matrix norm, but it is not the matrix norm induced by the Euclidean vector norm.

7.2 Eigenvalues, Spectral Radius, and Convergent Matrices

Definition 7.2.1. A nonzero vector $\boldsymbol{x}$ is an *eigenvector* of A associated with the *eigenvalue* λ if

$$A\boldsymbol{x} = \lambda \boldsymbol{x}.$$

Equivalently,

$$(A - \lambda I)\boldsymbol{x} = 0,$$

so eigenvalues satisfy $\det(A - \lambda I) = 0$.

Definition 7.2.2. The *spectral radius* of A is

$$\rho(A) = \max_{\lambda \in \sigma(A)} |\lambda| .$$

For every induced matrix norm,

$$\rho(A) \leq \|A\| .$$

Proof. Let $A\boldsymbol{x} = \lambda \boldsymbol{x}$ with $\|\boldsymbol{x}\| = 1$. Then

$$|\lambda| = \|\lambda \boldsymbol{x}\| = \|A\boldsymbol{x}\| \leq \|A\| .$$

Taking the maximum over all eigenvalues gives $\rho(A) \leq \|A\|$. □

If $A = A^T$, then

$$\|A\|_2 = \rho(A).$$

Definition 7.2.3. A square matrix A is called a *convergent matrix* if $A^k \to 0$ as $k \to \infty$.

Theorem 7.2.4. For $A \in \mathbb{R}^{n\times n}$, the following statements are equivalent:

1. $A^k \to 0$;
2. $\left\|A^k\right\| \to 0$ for some induced norm;
3. $\left\|A^k\right\| \to 0$ for every induced norm;
4. $\rho(A) < 1$;
5. $A^k\boldsymbol{x} \to 0$ for every $\boldsymbol{x} \in \mathbb{R}^n$.

The key criterion for iterative methods is therefore

$$\boxed{A^k \to 0 \quad \Longleftrightarrow \quad \rho(A) < 1.}$$

Theorem 7.2.5. If $\rho(T) < 1$, then $I - T$ is nonsingular and

$$(I - T)^{-1} = \sum_{k=0}^{\infty} T^k.$$

Proof. If $(I - T)\boldsymbol{x} = 0$ for some nonzero $\boldsymbol{x}$, then $T\boldsymbol{x} = \boldsymbol{x}$, so 1 is an eigenvalue of T, contradicting $\rho(T) < 1$. Hence $I - T$ is nonsingular.
Let

$$S_n = I + T + \cdots + T^n.$$

Then

$$(I - T)S_n = I - T^{n+1}.$$

Since $\rho(T) < 1$, $T^{n+1} \to 0$. Therefore

$$(I - T)\left(\lim_{n\to\infty} S_n\right) = I,$$

which gives the result. □

Consequently, the fixed-point equation

$$\boldsymbol{x} = T\boldsymbol{x} + \boldsymbol{c}$$

has a unique solution whenever $\rho(T) < 1$.

7.3 Residuals and Condition Numbers

Suppose $A\boldsymbol{x} = \boldsymbol{b}$, where A is nonsingular. For an approximate solution $\widetilde{\boldsymbol{x}}$, define the residual

$$\boldsymbol{r} = \boldsymbol{b} - A\widetilde{\boldsymbol{x}}.$$

If $\boldsymbol{x}$ is the exact solution and $\boldsymbol{e} = \boldsymbol{x} - \widetilde{\boldsymbol{x}}$, then

$$A\boldsymbol{e} = \boldsymbol{r}, \qquad \boldsymbol{e} = A^{-1}\boldsymbol{r}.$$

Hence

$$\|\boldsymbol{e}\| \le \left\|A^{-1}\right\| \|\boldsymbol{r}\|.$$

Since $\boldsymbol{b} = A\boldsymbol{x}$, $\|\boldsymbol{b}\| \le \|A\| \|\boldsymbol{x}\|$, so

$$\frac{\|\boldsymbol{e}\|}{\|\boldsymbol{x}\|} \le \|A\| \left\|A^{-1}\right\| \frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|}.$$

Definition 7.3.1. The *condition number* of a nonsingular matrix A with respect to a matrix norm is

$$\kappa(A) = \|A\| \left\|A^{-1}\right\|.$$

Since

$$1 = \|I\| = \left\|AA^{-1}\right\| \leq \|A\| \left\|A^{-1}\right\|,$$

we have $\kappa(A) \geq 1$.
If $A = A^T$ is symmetric and nonsingular, then for the 2-norm,

$$\kappa_2(A) = \frac{\max_i |\lambda_i(A)|}{\min_i |\lambda_i(A)|}.$$

A matrix with $\kappa(A)$ close to 1 is well conditioned. A very large condition number indicates an ill-conditioned system: small residuals or small perturbations in the data may correspond to relatively large errors in the solution.

7.4 Stationary Iterations and the Jacobi Method

A linear system can be rewritten as a fixed-point problem

$$A\boldsymbol{x} = \boldsymbol{b} \quad \Longleftrightarrow \quad \boldsymbol{x} = T\boldsymbol{x} + \boldsymbol{c}.$$

A stationary iterative method then has the form

$$\boxed{\boldsymbol{x}^{(k+1)} = T\boldsymbol{x}^{(k)} + \boldsymbol{c}.}$$

If $\boldsymbol{x}$ is the exact fixed point, define $\boldsymbol{e}^{(k)} = \boldsymbol{x} - \boldsymbol{x}^{(k)}$. Then

$$\boldsymbol{e}^{(k+1)} = T\boldsymbol{e}^{(k)} = T^{k+1}\boldsymbol{e}^{(0)}.$$

Therefore the iteration converges for every initial vector if and only if $T^k \to 0$, equivalently if and only if

$$\boxed{\rho(T) < 1}.$$

For the Jacobi method, write

$$A = D - L - U,$$

where D is the diagonal part of A, while $-L$ and $-U$ are the strictly lower and upper triangular parts.
The i-th equation is

$$a_{ii}x_i = b_i - \sum_{j \neq i} a_{ij}x_j.$$

Using only values from iteration k, Jacobi iteration is

$$\boxed{x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)} \right), \qquad i = 1, \ldots, n.}$$

In matrix form,

$$\boldsymbol{x}^{(k+1)} = D^{-1}(L + U)\boldsymbol{x}^{(k)} + D^{-1}\boldsymbol{b}.$$

Thus

$$T_J = D^{-1}(L + U), \qquad \boldsymbol{c}_J = D^{-1}\boldsymbol{b}.$$

7.5 Gauss–Seidel Method and Convergence Estimates

The Gauss–Seidel method improves the Jacobi iteration by immediately using newly computed components.
Thus

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right).$$

Using $A = D - L - U$,

$$(D - L)\boldsymbol{x}^{(k+1)} = U\boldsymbol{x}^{(k)} + \boldsymbol{b},$$

so

$$\boxed{\boldsymbol{x}^{(k+1)} = T_G \boldsymbol{x}^{(k)} + \boldsymbol{c}_G,}$$

where

$$T_G = (D - L)^{-1} U, \qquad \boldsymbol{c}_G = (D - L)^{-1} \boldsymbol{b}.$$

In many practical problems, Gauss–Seidel converges faster than Jacobi, although this is not true for every matrix.

Theorem 7.5.1. Let

$$\boldsymbol{x}^{(k+1)} = T\boldsymbol{x}^{(k)} + \boldsymbol{c}.$$

The iteration converges to the unique solution for every initial vector if and only if $\rho(T) < 1$.

If an induced matrix norm satisfies $\|T\| < 1$, then

$$\left\| \boldsymbol{x} - \boldsymbol{x}^{(k)} \right\| \leq \|T\|^k \left\| \boldsymbol{x} - \boldsymbol{x}^{(0)} \right\|.$$

Moreover,

$$\left\| \boldsymbol{x} - \boldsymbol{x}^{(k)} \right\| \leq \frac{\|T\|^k}{1 - \|T\|} \left\| \boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)} \right\|.$$

Proof. Since

$$\boldsymbol{x} - \boldsymbol{x}^{(k)} = T^k \left(\boldsymbol{x} - \boldsymbol{x}^{(0)} \right),$$

the first estimate follows immediately.
Also,

$$\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)} = T^k \left(\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)} \right).$$

For $m > k$,

$$\begin{aligned} \left\| \boldsymbol{x}^{(m)} - \boldsymbol{x}^{(k)} \right\| &\leq \sum_{j=k}^{m-1} \left\| \boldsymbol{x}^{(j+1)} - \boldsymbol{x}^{(j)} \right\| \\ &\leq \sum_{j=k}^{m-1} \|T\|^j \left\| \boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)} \right\|. \end{aligned}$$

Letting $m \to \infty$ gives

$$\left\| \boldsymbol{x} - \boldsymbol{x}^{(k)} \right\| \leq \frac{\|T\|^k}{1 - \|T\|} \left\| \boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)} \right\|.$$

□

Theorem 7.5.2. If A is strictly diagonally dominant, then both Jacobi and Gauss–Seidel iterations converge to the unique solution of $A\boldsymbol{x} = \boldsymbol{b}$ for every initial vector.

For Jacobi,

$$\|T_J\|_\infty = \max_i \frac{\sum_{j\neq i} |a_{ij}|}{|a_{ii}|} < 1,$$

so $\rho(T_J) \leq \|T_J\|_\infty < 1$.

Remark 7.5.3. The asymptotic convergence speed of a stationary iteration is governed by the spectral radius of its iteration matrix. Roughly,

$$\left\|\boldsymbol{e}^{(k)}\right\| \approx \rho(T)^k \left\|\boldsymbol{e}^{(0)}\right\|$$

for large k. Thus a smaller spectral radius generally indicates faster convergence.

7.6 Successive Over-Relaxation

The *Successive Over-Relaxation* method modifies Gauss–Seidel by introducing a relaxation parameter ω.
Let $x_i^{GS,(k+1)}$ denote the value produced by a Gauss–Seidel update. SOR defines

$$x_i^{(k+1)} = x_i^{(k)} + \omega\left(x_i^{GS,(k+1)} - x_i^{(k)}\right).$$

Equivalently,

$$\boxed{x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right).}$$

Using $A = D - L - U$,

$$(D-\omega L)\boldsymbol{x}^{(k+1)} = \Big[(1-\omega)D + \omega U\Big]\boldsymbol{x}^{(k)} + \omega\boldsymbol{b}.$$

Therefore the SOR iteration matrix is

$$\boxed{T_\omega = (D-\omega L)^{-1}\Big[(1-\omega)D + \omega U\Big].}$$

The parameter has the following interpretation:

$$\begin{aligned} 0 < \omega < 1 &\quad \text{under-relaxation},\\ \omega = 1 &\quad \text{Gauss–Seidel},\\ \omega > 1 &\quad \text{over-relaxation}. \end{aligned}$$

Theorem 7.6.1. If $a_{ii} \neq 0$, then

$$\rho(T_\omega) \geq |\omega - 1|.$$

Consequently, a necessary condition for convergence of SOR is

$$0 < \omega < 2.$$

Theorem 7.6.2. If A is symmetric positive definite and $0 < \omega < 2$, then SOR converges for every initial vector.

For appropriate matrix classes, especially SPD tridiagonal systems, a suitable choice of ω can substantially accelerate Gauss–Seidel convergence.

Remark 7.6.3. The purpose of SOR is not to define an entirely different fixed-point problem, but to modify the Gauss–Seidel correction. If the unrelaxed update moves too slowly toward the solution, over-relaxation extrapolates beyond the Gauss–Seidel value; if the update is too aggressive, under-relaxation reduces the step.

Chapter 8

Eigenvalue Problems and Matrix Decompositions

8.1 Eigenvalue Problems and Localization

Consider the eigenvalue problem

$$A\boldsymbol{v} = \lambda \boldsymbol{v}, \qquad \boldsymbol{v} \neq 0.$$

If $\boldsymbol{v}$ is an eigenvector associated with λ, then $c\boldsymbol{v}$ is also an eigenvector for every nonzero scalar c.
The eigenvalues of $A \in \mathbb{R}^{n\times n}$ are the roots of the characteristic equation

$$p_n(\lambda) = \det(A - \lambda I) = 0.$$

The characteristic polynomial has degree n. Its coefficients contain basic spectral information; in particular, the sum of the eigenvalues is $\operatorname{tr}(A)$, and their product is $\det A$.

Definition 8.1.1. The *algebraic multiplicity* of an eigenvalue λ is its multiplicity as a root of the characteristic polynomial. The *geometric multiplicity* is the dimension of its eigenspace $\ker(A - \lambda I)$.

The geometric multiplicity is no larger than the algebraic multiplicity.
If the total geometric multiplicity is n, then A has n linearly independent eigenvectors and is diagonalizable:

$$V^{-1}AV = \Lambda.$$

If $A = A^T$, then A has n orthonormal eigenvectors. Moreover, eigenvectors corresponding to distinct eigenvalues are orthogonal.

Proof. Suppose $A\boldsymbol{v}_1 = \lambda_1 \boldsymbol{v}_1$ and $A\boldsymbol{v}_2 = \lambda_2 \boldsymbol{v}_2$, where $\lambda_1 \neq \lambda_2$. Since $A = A^T$,

$$\lambda_1 \boldsymbol{v}_1^T \boldsymbol{v}_2 = (A\boldsymbol{v}_1)^T \boldsymbol{v}_2 = \boldsymbol{v}_1^T A \boldsymbol{v}_2 = \lambda_2 \boldsymbol{v}_1^T \boldsymbol{v}_2.$$

Hence $(\lambda_1 - \lambda_2)\boldsymbol{v}_1^T \boldsymbol{v}_2 = 0$, so $\boldsymbol{v}_1^T \boldsymbol{v}_2 = 0$. □

Thus every real symmetric matrix admits an orthogonal diagonalization

$$Q^T A Q = \Lambda.$$

Theorem 8.1.2 (Gershgorin Circle Theorem)**.** Let $A = (a_{ij}) \in \mathbb{C}^{n\times n}$. Define

$$R_i = \left\{ z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{j\neq i} |a_{ij}| \right\}.$$

Then every eigenvalue of A lies in

$$R = \bigcup_{i=1}^{n} R_i.$$

Proof. Let $A\boldsymbol{x} = \lambda \boldsymbol{x}$, and choose k such that $|x_k| = \max_i |x_i| > 0$. The k-th equation gives

$$(\lambda - a_{kk})x_k = \sum_{j\neq k} a_{kj} x_j.$$

Therefore

$$|\lambda - a_{kk}| \leq \sum_{j\neq k} |a_{kj}| \frac{|x_j|}{|x_k|} \leq \sum_{j\neq k} |a_{kj}|.$$

Hence $\lambda \in R_k$. □

Remark 8.1.3. If a union of k Gershgorin disks is disjoint from all remaining disks, then that union contains exactly k eigenvalues, counted with algebraic multiplicity.

8.2 Power Methods, Shifts, and Rayleigh Quotients

Assume the eigenvalues are ordered so that

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|.$$

Then λ_1 is the *dominant eigenvalue.*

8.2.1 Power Method

Choose a nonzero initial vector $\boldsymbol{x}^{(0)}$, and define

$$\boldsymbol{y}^{(k)} = A\boldsymbol{x}^{(k-1)}, \qquad \boldsymbol{x}^{(k)} = \frac{\boldsymbol{y}^{(k)}}{\|\boldsymbol{y}^{(k)}\|}.$$

Suppose first that A is diagonalizable and $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$ are linearly independent eigenvectors. Write

$$\boldsymbol{x}^{(0)} = \sum_{j=1}^{n} c_j \boldsymbol{v}_j.$$

Then

$$A^k \boldsymbol{x}^{(0)} = \lambda_1^k \left[c_1 \boldsymbol{v}_1 + \sum_{j=2}^{n} c_j \left(\frac{\lambda_j}{\lambda_1} \right)^k \boldsymbol{v}_j \right].$$

If $c_1 \neq 0$, then

$$\left(\frac{\lambda_j}{\lambda_1} \right)^k \to 0$$

for every $j \geq 2$. Therefore, after normalization, $\boldsymbol{x}^{(k)}$ converges to the eigendirection associated with λ_1.

Theorem 8.2.1. If the dominant eigenvalue of A has algebraic multiplicity one, then for almost every initial vector $\boldsymbol{x}^{(0)}$, the normalized power iteration converges to an eigenvector associated with the dominant eigenvalue.

The asymptotic convergence rate is governed by

$$O\left(\left| \frac{\lambda_2}{\lambda_1} \right|^k \right).$$

Thus, if $|\lambda_2/\lambda_1|$ is close to 1, convergence is slow.

8.2.2 Inverse Power Method with Shift

Suppose we want the eigenvalue of A closest to a prescribed number μ. The eigenvalues of $(A - \mu I)^{-1}$ are

$$\frac{1}{\lambda_j - \mu}.$$

Hence the eigenvalue of A closest to μ corresponds to the dominant eigenvalue of $(A - \mu I)^{-1}$.

Rather than computing the inverse explicitly, at each step solve

$$(A - \mu I)\boldsymbol{y}^{(k)} = \boldsymbol{x}^{(k-1)}$$

and normalize

$$\boldsymbol{x}^{(k)} = \frac{\boldsymbol{y}^{(k)}}{\|\boldsymbol{y}^{(k)}\|}.$$

The convergence factor is determined by the ratios

$$\left|\frac{\lambda_1 - \mu}{\lambda_j - \mu}\right|^k,$$

where λ_1 denotes the eigenvalue closest to μ. A good shift can therefore make the convergence substantially faster.

8.2.3 Rayleigh Quotient Iteration

For a real symmetric matrix A, define the Rayleigh quotient

$$R(A, \boldsymbol{x}) = \frac{\boldsymbol{x}^T A \boldsymbol{x}}{\boldsymbol{x}^T \boldsymbol{x}}, \qquad \boldsymbol{x} \neq 0.$$

If $\boldsymbol{x} = \sum_j c_j \boldsymbol{v}_j$, where $\boldsymbol{v}_j$ are orthonormal eigenvectors, then

$$R(A, \boldsymbol{x}) = \frac{\sum_j \lambda_j c_j^2}{\sum_j c_j^2}.$$

Therefore

$$\lambda_{\min} \leq R(A, \boldsymbol{x}) \leq \lambda_{\max}.$$

Rayleigh quotient iteration updates the shift dynamically. Choose $\left\|\boldsymbol{x}^{(0)}\right\|_2 = 1$ and set $\mu^{(0)} = (\boldsymbol{x}^{(0)})^T A \boldsymbol{x}^{(0)}$. For $k = 1, 2, \ldots,$

1. solve
$$(A - \mu^{(k-1)} I)\boldsymbol{y}^{(k)} = \boldsymbol{x}^{(k-1)};$$
2. normalize
$$\boldsymbol{x}^{(k)} = \frac{\boldsymbol{y}^{(k)}}{\|\boldsymbol{y}^{(k)}\|_2};$$
3. update
$$\mu^{(k)} = (\boldsymbol{x}^{(k)})^T A \boldsymbol{x}^{(k)}.$$

For symmetric matrices, Rayleigh quotient iteration has cubic local convergence under the usual nondegeneracy assumptions.
A disadvantage is that it is not always possible to determine in advance which eigenvalue the iteration will approach.

Theorem 8.2.2. Let $A = A^T$, let λ_j be an eigenvalue of A, and suppose $\|\boldsymbol{x}\|_2 = 1$. If

$$\|A\boldsymbol{x} - \lambda \boldsymbol{x}\|_2 < \varepsilon,$$

then

$$\min_j |\lambda - \lambda_j| < \varepsilon.$$

Proof. Expand $\boldsymbol{x} = \sum_j c_j \boldsymbol{v}_j$ in an orthonormal eigenbasis. Then

$$\|A\boldsymbol{x} - \lambda \boldsymbol{x}\|_2^2 = \sum_j (\lambda_j - \lambda)^2 c_j^2.$$

Since $\sum_j c_j^2 = 1$,

$$\|A\boldsymbol{x} - \lambda \boldsymbol{x}\|_2^2 \geq \left(\min_j |\lambda_j - \lambda|\right)^2.$$

The result follows. □

8.2.4 Deflation

Suppose A has a normalized eigenpair $(\lambda_1, \boldsymbol{v}_1)$, with $\boldsymbol{v}_1^T \boldsymbol{v}_1 = 1$. Define

$$B = A - \lambda_1 \boldsymbol{v}_1 \boldsymbol{v}_1^T.$$

Then

$$B\boldsymbol{v}_1 = 0.$$

For a symmetric matrix, if $A\boldsymbol{v}_j = \lambda_j \boldsymbol{v}_j$ with $j \neq 1$, orthogonality gives $\boldsymbol{v}_1^T \boldsymbol{v}_j = 0$, and hence

$$B\boldsymbol{v}_j = A\boldsymbol{v}_j - \lambda_1 \boldsymbol{v}_1 \boldsymbol{v}_1^T \boldsymbol{v}_j = \lambda_j \boldsymbol{v}_j.$$

Thus deflation removes the already computed eigenvalue while preserving the remaining eigenpairs.

8.3 Householder Transformations and Matrix Reduction

Directly applying eigenvalue iterations to a dense matrix can be expensive. A standard first step is to reduce the matrix to a simpler form using orthogonal transformations.

Definition 8.3.1. Let $\boldsymbol{w} \in \mathbb{R}^n$ satisfy $\boldsymbol{w}^T \boldsymbol{w} = 1$. The matrix

$$H = I - 2\boldsymbol{w}\boldsymbol{w}^T$$

is called a *Householder transformation*.

It is both symmetric and orthogonal:

$$H^T = H, \qquad H^T H = I.$$

Proof. Since

$$H^T = I - 2\boldsymbol{w}\boldsymbol{w}^T = H,$$

and $\boldsymbol{w}^T \boldsymbol{w} = 1$,

$$H^2 = (I - 2\boldsymbol{w}\boldsymbol{w}^T)^2 = I.$$

Thus $H^{-1} = H = H^T$. □

Geometrically, H is a reflection across the hyperplane orthogonal to $\boldsymbol{w}$.

Theorem 8.3.2. Let $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ satisfy $\|\boldsymbol{x}\|_2 = \|\boldsymbol{y}\|_2$. Then there exists a Householder transformation H such that $H\boldsymbol{x} = \boldsymbol{y}$.

For numerical elimination, a particularly useful choice maps $\boldsymbol{x}$ to a multiple of the first coordinate vector. Let

$$\sigma = \operatorname{sign}(x_1) \|\boldsymbol{x}\|_2$$

when $x_1 \neq 0$, and choose

$$\boldsymbol{u} = \boldsymbol{x} + \sigma \boldsymbol{e}_1.$$

Then the corresponding Householder matrix maps $\boldsymbol{x}$ to $-\sigma \boldsymbol{e}_1$.
The sign is chosen to avoid catastrophic cancellation in forming $\boldsymbol{u}$.

8.3.1 Reduction to Hessenberg and Tridiagonal Form

For a general matrix A, successive Householder transformations produce an orthogonally similar upper Hessenberg matrix

$$H = Q^T A Q,$$

where

$$H = \begin{pmatrix} * & * & * & \cdots & * \\ * & * & * & \cdots & * \\ 0 & * & * & \cdots & * \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & * & * \end{pmatrix}.$$

If $A = A^T$, symmetry is preserved under orthogonal similarity, so the Hessenberg form is also symmetric. Hence it must be tridiagonal:

$$T = Q^T A Q = \begin{pmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \end{pmatrix}.$$

The reduction costs $O(n^3)$ operations, but it is performed only once. Subsequent QR iterations are much cheaper on Hessenberg or tridiagonal matrices.

Remark 8.3.3. Householder transformations simplify a matrix while preserving its eigenvalues. They do not, by themselves, diagonalize a general matrix.

8.4 QR Factorization and the QR Eigenvalue Algorithm

Definition 8.4.1. A QR factorization of $A \in \mathbb{R}^{m \times n}$ is a factorization

$$A = QR,$$

where Q has orthonormal columns and R is upper triangular.

QR factorization can be constructed using Householder transformations or Givens rotations.

8.4.1 Givens Rotations

In two dimensions, define

$$G(\theta) = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}.$$

The transformation preserves Euclidean length.
Given a, b, choose

$$\cos\theta = \frac{a}{\sqrt{a^2+b^2}}, \qquad \sin\theta = \frac{b}{\sqrt{a^2+b^2}}.$$

Then

$$G(\theta)\begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} \sqrt{a^2+b^2} \\ 0 \end{pmatrix}.$$

In higher dimensions, a Givens rotation acts only on two selected coordinates and leaves the remaining coordinates unchanged. It can therefore eliminate one matrix entry at a time.

Successively applying Givens rotations below the main diagonal produces

$$P_s \cdots P_2 P_1 A = R,$$

where R is upper triangular. Since every P_j is orthogonal,

$$A = QR, \qquad Q = P_1^T P_2^T \cdots P_s^T.$$

8.4.2 QR Iteration

Let $A^{(0)} = A$. At iteration k, compute

$$A^{(k)} = Q^{(k)} R^{(k)}$$

and then reverse the factors:

$$\boxed{A^{(k+1)} = R^{(k)} Q^{(k)}.}$$

Since

$$R^{(k)} = (Q^{(k)})^T A^{(k)},$$

we have

$$A^{(k+1)} = (Q^{(k)})^T A^{(k)} Q^{(k)}.$$

Thus every QR step is an orthogonal similarity transformation, so $A^{(k+1)}$ has the same eigenvalues as $A^{(k)}$.
If the initial matrix is symmetric, every $A^{(k)}$ remains symmetric. If it is also tridiagonal, the QR iteration preserves tridiagonal structure.
Under appropriate assumptions, the off-diagonal entries tend to zero and

$$A^{(k)} \longrightarrow \Lambda,$$

so the diagonal entries converge to eigenvalues of A.

Remark 8.4.2. In practical implementations, one does not explicitly form the full orthogonal matrix $Q^{(k)}$ at every step. The orthogonal transformations are applied directly to the matrix so that the Hessenberg or tridiagonal structure is preserved.

8.5 Singular Value Decomposition

Eigenvalue decomposition applies most naturally to square matrices. Singular value decomposition extends the same spectral philosophy to arbitrary rectangular matrices.

Theorem 8.5.1 (Singular Value Decomposition)**.** For every $A \in \mathbb{R}^{m \times n}$, there exist orthogonal matrices $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ such that

$$\boxed{A = U \Sigma V^T,}$$

where

$$\Sigma = \begin{pmatrix} \sigma_1 & & & & \\ & \sigma_2 & & & \\ & & \ddots & & \\ & & & \sigma_r & \\ & & & & 0 \end{pmatrix} \in \mathbb{R}^{m \times n},$$

and

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

The numbers σ_i are the nonzero singular values of A.

From $A = U\Sigma V^T$,

$$AA^T = U\Sigma\Sigma^T U^T$$

and

$$A^T A = V\Sigma^T\Sigma V^T.$$

Therefore the columns of U are eigenvectors of AA^T, the columns of V are eigenvectors of $A^T A$, and

$$\sigma_i^2$$

are the corresponding nonzero eigenvalues.

Theorem 8.5.2. For $A \in \mathbb{R}^{m\times n}$,

1. $A^T A$ and AA^T are symmetric positive semidefinite;
2. $\ker(A) = \ker(A^T A)$;
3. $\operatorname{rank}(A) = \operatorname{rank}(A^T A)$;
4. $A^T A$ and AA^T have the same nonzero eigenvalues.

Proof. For every $\boldsymbol{x}$,

$$\boldsymbol{x}^T A^T A\boldsymbol{x} = \|A\boldsymbol{x}\|_2^2 \geq 0,$$

so $A^T A$ is positive semidefinite. The argument for AA^T is identical.
Moreover,

$$A^T A\boldsymbol{x} = 0 \quad\Longrightarrow\quad \boldsymbol{x}^T A^T A\boldsymbol{x} = 0 \quad\Longrightarrow\quad A\boldsymbol{x} = 0,$$

and the reverse implication is immediate. Hence $\ker(A) = \ker(A^T A)$, which also yields equality of ranks.
Finally, if $A^T A\boldsymbol{v} = \sigma^2\boldsymbol{v}$ with $\sigma \neq 0$, define $\boldsymbol{u} = A\boldsymbol{v}/\sigma$. Then $AA^T\boldsymbol{u} = \sigma^2\boldsymbol{u}$, showing that the nonzero spectra coincide. □

A practical construction of the SVD proceeds as follows.

1. Diagonalize $A^T A$. Let $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$ be an orthonormal eigenbasis with eigenvalues
$$\sigma_1^2 \geq \sigma_2^2 \geq \cdots \geq \sigma_r^2 > 0.$$
2. Form
$$V = \begin{pmatrix} \boldsymbol{v}_1 & \cdots & \boldsymbol{v}_n \end{pmatrix}.$$
3. For $i = 1, \ldots, r$, define
$$\boldsymbol{u}_i = \frac{1}{\sigma_i} A\boldsymbol{v}_i.$$
4. Complete $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$ to an orthonormal basis of $\mathbb{R}^m$, for example by Gram–Schmidt, and form U.

The vectors $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$ are orthonormal because

$$\boldsymbol{u}_i^T \boldsymbol{u}_j = \frac{1}{\sigma_i\sigma_j} \boldsymbol{v}_i^T A^T A\boldsymbol{v}_j = \delta_{ij}.$$

The columns $\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n$ form a basis of $\ker(A^T A) = \ker(A)$.

Remark 8.5.3. One may theoretically begin with the eigendecomposition of AA^T, but if $m \gg n$, then AA^T is an $m \times m$ matrix while $A^T A$ is only $n \times n$. In such a case, starting with $A^T A$ is computationally preferable.

8.6 Applications of the Singular Value Decomposition

The SVD has several important applications in numerical computation.

8.6.1 Low-Rank Approximation and Data Compression

The SVD can be written as

$$A = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

If the first k singular values dominate the rest, define

$$A_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

Then A_k provides a low-rank approximation of A.
Instead of storing all mn entries of A, one stores the first k left singular vectors, the first k right singular vectors, and the corresponding singular values. When $k \ll \min\{m, n\}$, this can substantially reduce storage.

8.6.2 Least-Squares Problems

Consider

$$\min_{\boldsymbol{x}} \|A\boldsymbol{x} - \boldsymbol{b}\|_2 .$$

If $A = U\Sigma V^T$, set

$$\boldsymbol{z} = V^T\boldsymbol{x}, \qquad \boldsymbol{c} = U^T\boldsymbol{b}.$$

Since orthogonal transformations preserve the Euclidean norm,

$$\|A\boldsymbol{x} - \boldsymbol{b}\|_2 = \|\Sigma\boldsymbol{z} - \boldsymbol{c}\|_2 .$$

For each nonzero singular value,

$$\sigma_i z_i = c_i,$$

so

$$z_i = \frac{c_i}{\sigma_i}, \qquad i = 1, \ldots, r.$$

The remaining components are free. A standard minimum-norm choice is to set them equal to zero.
Thus the SVD provides a stable way to solve least-squares problems, particularly when A is rank deficient or nearly rank deficient.

8.6.3 Principal Component Analysis

Suppose data vectors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N \in \mathbb{R}^d$ are assembled as

$$X = \begin{pmatrix} \boldsymbol{x}_1 & \boldsymbol{x}_2 & \cdots & \boldsymbol{x}_N \end{pmatrix}.$$

Let

$$X = U\Sigma V^T.$$

To reduce the data from dimension d to $k < d$, choose the first k left singular vectors and define

$$P = \begin{pmatrix} \boldsymbol{u}_1^T \\ \vdots \\ \boldsymbol{u}_k^T \end{pmatrix}.$$

The reduced data are

$$Y = PX.$$

Since the columns of U are orthonormal,

$$PU = \begin{pmatrix} I_k & 0 \end{pmatrix},$$

so

$$YY^T = PXX^TP^T$$

is diagonal in the principal-component coordinates.
Thus PCA retains the directions associated with the largest singular values and discards directions carrying less variation.

Chapter 9

Least-Squares Approximation and Orthogonal Polynomials

9.1 Data Fitting and Least-Squares Problems

Suppose a set of data points

$$\{(x_i, y_i)\}_{i=1}^{n}$$

is given, where ideally $y_i = f(x_i)$. The goal of data fitting is to find a simple function that approximates the observed data.
For a linear model,

$$y = a_1 x + a_0,$$

the residual at the point x_i is

$$e_i = a_1 x_i + a_0 - y_i.$$

Different norms lead to different fitting criteria.
The L^∞-type fitting problem minimizes the largest residual:

$$\min_{a_0, a_1} \max_{1 \le i \le n} \left| a_1 x_i + a_0 - y_i \right|.$$

The L^1-type fitting problem minimizes the total absolute deviation:

$$\min_{a_0, a_1} \sum_{i=1}^{n} \left| a_1 x_i + a_0 - y_i \right|.$$

The most common choice is the L^2-type fitting problem:

$$\min_{a_0, a_1} \sum_{i=1}^{n} \left| a_1 x_i + a_0 - y_i \right|^2.$$

Define

$$F(a_0, a_1) = \frac{1}{2} \sum_{i=1}^{n} (a_1 x_i + a_0 - y_i)^2.$$

At a minimizer,

$$\frac{\partial F}{\partial a_0} = 0, \qquad \frac{\partial F}{\partial a_1} = 0.$$

Therefore

$$n a_0 + \left(\sum_{i=1}^{n} x_i \right) a_1 = \sum_{i=1}^{n} y_i$$

and

$$\left(\sum_{i=1}^{n} x_i \right) a_0 + \left(\sum_{i=1}^{n} x_i^2 \right) a_1 = \sum_{i=1}^{n} x_i y_i.$$

Solving these equations gives

$$a_1 = \frac{n \sum_i x_i y_i - \left(\sum_i x_i \right) \left(\sum_i y_i \right)}{n \sum_i x_i^2 - \left(\sum_i x_i \right)^2},$$

and

$$a_0 = \frac{1}{n} \left(\sum_i y_i - a_1 \sum_i x_i \right).$$

Thus the least-squares line is the line that minimizes the sum of squared residuals rather than forcing the fitted function to pass through every data point.

9.2 Normal Equations and Weighted Least Squares

The linear least-squares problem can be written in matrix form as

$$\min_{\boldsymbol{x}} \|A\boldsymbol{x} - \boldsymbol{b}\|_2 .$$

Define

$$F(\boldsymbol{x}) = \frac{1}{2} \|A\boldsymbol{x} - \boldsymbol{b}\|_2^2 .$$

Then

$$F(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T A^T A\boldsymbol{x} - \boldsymbol{x}^T A^T \boldsymbol{b} + \frac{1}{2}\boldsymbol{b}^T \boldsymbol{b}.$$

Hence

$$\nabla F(\boldsymbol{x}) = A^T A\boldsymbol{x} - A^T \boldsymbol{b}.$$

Therefore any least-squares solution satisfies the *normal equations*

$$\boxed{A^T A\boldsymbol{x} = A^T \boldsymbol{b}.}$$

This condition also has a geometric interpretation. Let

$$\boldsymbol{r} = \boldsymbol{b} - A\boldsymbol{x}.$$

At the least-squares solution,

$$A^T \boldsymbol{r} = 0.$$

Thus the residual is orthogonal to the column space of A:

$$\boldsymbol{r} \perp \mathcal{C}(A).$$

Equivalently, write

$$\boldsymbol{b} = \boldsymbol{u} + \boldsymbol{v},$$

where $\boldsymbol{u} \in \mathcal{C}(A)$ and $\boldsymbol{v} \in \mathcal{N}(A^T)$. Then the minimum is attained when $A\boldsymbol{x} = \boldsymbol{u}$, and the residual is precisely $\boldsymbol{v}$.
If A has full column rank, then $A^T A$ is symmetric positive definite, so the least-squares solution is unique.

9.2.1 Weighted Least Squares

Sometimes different observations should carry different importance. Let $w_i > 0$ be weights and define

$$F(\boldsymbol{x}) = \frac{1}{2} \sum_{i=1}^{m} w_i \left| (A\boldsymbol{x})_i - b_i \right|^2 .$$

Let

$$W = \operatorname{diag}(w_1, \ldots, w_m).$$

Then

$$F(\boldsymbol{x}) = \frac{1}{2}(A\boldsymbol{x} - \boldsymbol{b})^T W (A\boldsymbol{x} - \boldsymbol{b}).$$

Differentiation gives

$$A^T W A\boldsymbol{x} = A^T W \boldsymbol{b}.$$

Thus the ordinary normal equations are replaced by the weighted normal equations.

9.2.2 Polynomial Least Squares

Suppose the data are approximated by a polynomial

$$P_m(x) = \sum_{k=0}^{m} a_k x^k.$$

Define

$$E(\boldsymbol{a}) = \frac{1}{2} \sum_{i=1}^{n} |P_m(x_i) - y_i|^2 .$$

The optimal coefficients satisfy

$$\frac{\partial E}{\partial a_j} = 0, \qquad j = 0, 1, \ldots, m.$$

Hence

$$\sum_{k=0}^{m} \left(\sum_{i=1}^{n} x_i^{j+k} \right) a_k = \sum_{i=1}^{n} x_i^j y_i, \qquad j = 0, \ldots, m.$$

Define

$$M_{jk} = \sum_{i=1}^{n} x_i^{j+k}, \qquad b_j = \sum_{i=1}^{n} x_i^j y_i.$$

Then

$$M\boldsymbol{a} = \boldsymbol{b}.$$

If the data points x_i are sufficiently distinct and the fitting problem has full rank, the coefficient system is nonsingular.

Remark 9.2.1. An exponential model can sometimes be transformed into a linear fitting problem. For example, if

$$y = be^{ax},$$

then taking logarithms gives

$$\ln y = ax + \ln b.$$

Thus one may first fit a straight line to $(x_i, \ln y_i)$ and then recover b from the intercept.

9.3 Continuous Least-Squares Approximation

The discrete least-squares principle extends naturally to approximation of functions.
By the Weierstrass approximation theorem, for every $f \in C[a, b]$ and every $\varepsilon > 0$, there exists a polynomial P_n such that

$$\max_{x \in [a,b]} |P_n(x) - f(x)| < \varepsilon.$$

Thus polynomials are dense in $C[a, b]$.
Instead of minimizing the maximum error, we may minimize the L^2-error. Let

$$P_n(x) = \sum_{k=0}^{n} a_k x^k.$$

Define

$$E(\boldsymbol{a}) = \frac{1}{2} \int_a^b |P_n(x) - f(x)|^2 \, dx.$$

At the minimizer,

$$\frac{\partial E}{\partial a_j} = 0, \qquad j = 0, \ldots, n.$$

Therefore

$$\int_a^b x^j \left(P_n(x) - f(x)\right)\, dx = 0.$$

Equivalently,

$$\sum_{k=0}^{n} \left(\int_a^b x^{j+k}\, dx \right) a_k = \int_a^b x^j f(x)\, dx.$$

This is again a system of normal equations, now in a function space.
More generally, suppose

$$\phi_0(x), \phi_1(x), \ldots, \phi_n(x)$$

are linearly independent functions on $[a, b]$, and approximate f by

$$P_n(x) = \sum_{k=0}^{n} c_k \phi_k(x).$$

Then

$$E = \frac{1}{2} \int_a^b |P_n(x) - f(x)|^2\, dx.$$

Differentiating with respect to c_j gives

$$\int_a^b \phi_j(x) \left(P_n(x) - f(x)\right)\, dx = 0.$$

Hence

$$\sum_{k=0}^{n} \left(\int_a^b \phi_j(x)\phi_k(x)\, dx \right) c_k = \int_a^b \phi_j(x) f(x)\, dx.$$

If the basis functions are orthogonal, the coefficient system becomes diagonal.

9.4 Orthogonal Bases and Gram–Schmidt Construction

Introduce the weighted inner product

$$(f, g)_w = \int_a^b w(x) f(x) g(x)\, dx,$$

where $w(x) > 0$.

Definition 9.4.1. A set of functions

$$\phi_0, \phi_1, \ldots, \phi_n$$

is *orthogonal* with respect to the weight w if

$$(\phi_j, \phi_k)_w = 0$$

whenever $j \neq k$.
It is *orthonormal* if, in addition,

$$(\phi_j, \phi_j)_w = 1.$$

If

$$P_n(x) = \sum_{j=0}^{n} c_j \phi_j(x)$$

is the weighted least-squares approximation of f, then orthogonality gives

$$\boxed{c_j = \frac{(\phi_j, f)_w}{(\phi_j, \phi_j)_w}.}$$

Thus orthogonal bases eliminate the need to solve a full system of normal equations. Orthogonal polynomial bases can be generated by a Gram–Schmidt process. Start with $\phi_0(x) = 1$. A first-degree polynomial may be written as

$$\phi_1(x) = x - \beta_1,$$

where

$$\beta_1 = \frac{(x, \phi_0)_w}{(\phi_0, \phi_0)_w}.$$

For higher degrees, orthogonal polynomials satisfy a three-term recurrence of the form

$$\phi_k(x) = (x - \beta_k)\phi_{k-1}(x) - \gamma_k \phi_{k-2}(x),$$

where

$$\beta_k = \frac{(x\phi_{k-1}, \phi_{k-1})_w}{(\phi_{k-1}, \phi_{k-1})_w}$$

and

$$\gamma_k = \frac{(x\phi_{k-1}, \phi_{k-2})_w}{(\phi_{k-2}, \phi_{k-2})_w}.$$

By induction, the resulting family is orthogonal with respect to the chosen weight function.

Theorem 9.4.2. Suppose $\phi_0, \phi_1, \ldots, \phi_n$ are orthogonal with respect to a positive weight function w on $[a, b]$, and $\deg \phi_k = k$. Then ϕ_k has k distinct real roots in (a, b).

Proof. Suppose ϕ_k has only $x_1, \ldots, x_\ell$ as roots in (a, b) with odd multiplicity, where $\ell < k$. Define

$$q(x) = \prod_{j=1}^{\ell} (x - x_j).$$

Then $\deg q < k$, so by orthogonality,

$$\int_a^b q(x)\phi_k(x)w(x)\,dx = 0.$$

However, by construction, $q(x)\phi_k(x)$ does not change sign on (a, b) and is not identically zero. Since $w(x) > 0$,

$$\int_a^b q(x)\phi_k(x)w(x)\,dx \neq 0,$$

a contradiction. Hence ϕ_k must have k distinct roots in (a, b). □

9.5 Legendre and Chebyshev Polynomials

9.5.1 Legendre Polynomials

On $[-1, 1]$ with weight $w(x) = 1$, the resulting orthogonal polynomials are the Legendre polynomials.
A normalized form is given by Rodrigues' formula

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n}(x^2 - 1)^n.$$

The Legendre polynomials satisfy the recurrence

$$(n + 1)P_{n+1}(x) = (2n + 1)xP_n(x) - nP_{n-1}(x).$$

The first few are

$$P_0(x) = 1, \qquad P_1(x) = x,$$
$$P_2(x) = \frac{1}{2}(3x^2 - 1), \qquad P_3(x) = \frac{1}{2}(5x^3 - 3x).$$

The family

$$\{P_n\}_{n=0}^{\infty}$$

is orthogonal on $[-1, 1]$:

$$\int_{-1}^{1} P_m(x)P_n(x)\,dx = 0$$

whenever $m \neq n$.
Moreover, P_n is even when n is even and odd when n is odd.

9.5.2 Chebyshev Polynomials

On $[-1, 1]$, consider the weight

$$w(x) = \frac{1}{\sqrt{1 - x^2}}.$$

The Chebyshev polynomial of degree n is

$$\boxed{T_n(x) = \cos(n \arccos x).}$$

Set $x = \cos\theta$. Then

$$T_n(x) = \cos(n\theta).$$

Using the trigonometric identity

$$\cos((n + 1)\theta) + \cos((n - 1)\theta) = 2\cos\theta\cos(n\theta),$$

we obtain the recurrence

$$\boxed{T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x).}$$

The first few Chebyshev polynomials are

$$T_0(x) = 1, \qquad T_1(x) = x,$$
$$T_2(x) = 2x^2 - 1, \qquad T_3(x) = 4x^3 - 3x,$$
$$T_4(x) = 8x^4 - 8x^2 + 1.$$

The Chebyshev polynomials are orthogonal with respect to the weight $w(x) = 1/\sqrt{1-x^2}$:

$$\int_{-1}^{1} \frac{T_m(x)T_n(x)}{\sqrt{1-x^2}}\,dx = 0$$

whenever $m \neq n$.
For $n \geq 1$, the leading coefficient of T_n is 2^{n-1}.
Also, T_{2m} is even and T_{2m+1} is odd.
The zeros of T_n satisfy

$$n \arccos x_k = \frac{(2k-1)\pi}{2},$$

so

$$\boxed{x_k = \cos\left(\frac{(2k-1)\pi}{2n}\right), \qquad k = 1, \ldots, n.}$$

These points are more densely distributed near the endpoints -1 and 1.
The extreme points satisfy

$$x_k = \cos\left(\frac{k\pi}{n}\right), \qquad k = 0, \ldots, n,$$

and

$$T_n(x_k) = (-1)^k.$$

Theorem 9.5.1 (Minimax Property of Chebyshev Polynomials)**.** Among all monic polynomials p_n of degree n,

$$\max_{x\in[-1,1]} |p_n(x)| \geq 2^{1-n}.$$

Equality is attained by

$$2^{1-n}T_n(x).$$

Proof. Suppose there exists a monic polynomial p_n such that

$$\max_{x\in[-1,1]} |p_n(x)| < 2^{1-n}.$$

Define

$$q(x) = 2^{1-n}T_n(x) - p_n(x).$$

The leading coefficients cancel, so $\deg q \leq n-1$.
At the extreme points $x_k = \cos(k\pi/n)$,

$$2^{1-n}T_n(x_k) = 2^{1-n}(-1)^k.$$

Since $|p_n(x_k)| < 2^{1-n}$, the values $q(x_k)$ alternate in sign. Therefore q has at least n distinct zeros between consecutive extreme points, contradicting $\deg q \leq n-1$. □

9.5.3 Chebyshev Nodes and Interpolation Error

Recall the interpolation error formula

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!}\omega_{n+1}(x),$$

where

$$\omega_{n+1}(x) = \prod_{j=0}^{n}(x - x_j).$$

To reduce the maximum interpolation error, one should choose the nodes so that

$$\max_{x\in[-1,1]} |\omega_{n+1}(x)|$$

is as small as possible.
Since ω_{n+1} is a monic polynomial of degree $n+1$, the minimax property shows that the optimal choice is obtained from a scaled Chebyshev polynomial. Therefore choose the interpolation nodes as the zeros of T_{n+1}:

$$\boxed{x_j = \cos\left(\frac{(2j+1)\pi}{2(n+1)}\right), \qquad j = 0, \ldots, n.}$$

For these nodes,

$$\max_{x\in[-1,1]} |\omega_{n+1}(x)| = 2^{-n}.$$

Hence

$$\boxed{\max_{x\in[-1,1]} |f(x) - P_n(x)| \le \frac{2^{-n}}{(n+1)!} \max_{x\in[-1,1]} \left|f^{(n+1)}(x)\right|.}$$

Thus Chebyshev nodes reduce the worst-case interpolation factor by clustering more nodes near the endpoints, where equally spaced interpolation is most likely to exhibit large oscillations.

Acknowledgements

I would like to express my sincere gratitude to Professor Jiang Yang for his teaching and for the handwritten lecture notes on which these notes are primarily based.
The organization and mathematical development of this document closely follow the ideas and progression presented in his handwritten notes. I have reorganized the material into a more systematic set of lecture notes, translated the original Chinese annotations into English, and added several supplementary explanations and details where appropriate.
I am especially grateful for Professor Yang's clear presentation of the underlying ideas in numerical methods for partial differential equations, including finite difference methods, variational and finite element methods, time discretization, stability analysis, and numerical methods for parabolic and hyperbolic equations. His lectures and notes provided the main conceptual framework for this document.
Any errors, omissions, or inaccuracies in these notes are my own.